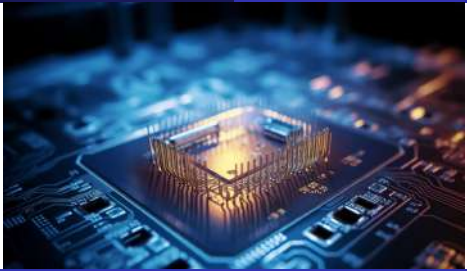


Introducción a la Arquitectura y Organización de Computadores

Alejandro Furfaro

21 de agosto de 2025

Contenido

1. Pioneros
   - Introducción histórica para empezar a entender algunos "*porque*"
2. Evolución
   - Vorágine evolutiva
   - La era del Silicio
   - Nace un paradigma de Arquitectura: RISC
3. Arquitectura y Organización
   - Introducción
4. Implementando la arquitectura
   - El Procesador
   - Implementación del Procesador
   - Pipeline
   - La memoria. El problema del acceso

1 Pioneros
- Introducción histórica para empezar a entender algunos "*porque*"

2 Evolución

3 Arquitectura y Organización

4 Implementando la arquitectura

¿Quien diseñó el primer computador de propósito general?

- ¿Fue Alan Turing durante los años 30?

¿Quien diseñó el primer computador de propósito general?

- ¿Fue Alan Turing durante los años 30?
- ¿Fue Von Newmann en los años 40?

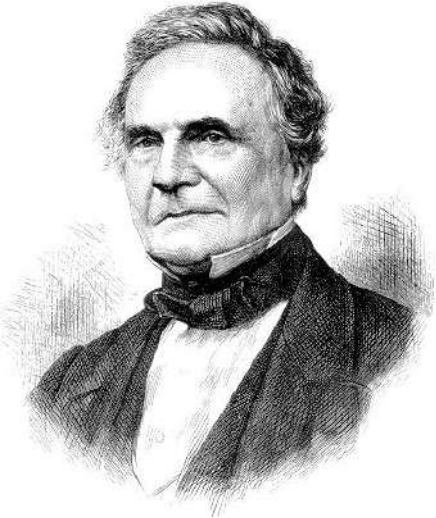
¿Quien diseñó el primer computador de propósito general?

- ¿Fue Alan Turing durante los años 30?
- ¿Fue Von Newmann en los años 40?
- ¿Fueron los alemanes durante la 2da. Guerra Mundial?

¿Quien diseñó el primer computador de propósito general?

- ¿Fue Alan Turing durante los años 30?
- ¿Fue Von Newmann en los años 40?
- ¿Fueron los alemanes durante la 2da. Guerra Mundial?
- .... Frío ....

Fue en el sigo XIX....



Charles Babbage 26/12/1791 - 18/10/1871

Máquina Analítica (1837)

Analitical Machine. Foto: ©De Bruno Barral (ByB), CC BY-SA 2.5,
https://commons.wikimedia.org/w/index.php?curid=6839854

Diferential Engine (14 de Junio de 1821)

Diferential Engine. Foto: ©De Carsten Ullrich - Trabajo propio, CC BY-SA 2.5, https://commons.wikimedia.org/w/index.php?curid=851017

Z3: (1941) Primer Computador Programable

Réplica del Computador Z3. Foto: ©De Venusianer, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=3632073

La Máquina de Turing

Alan Turing. Foto: ©De Desconocido - http://www.turingarchive.org/viewer/?id=521&title=4, Dominio público, https://commons.wikimedia.org/w/index.php?curid=22828488

- Hasta que Alan Turing enunció en 1936 su modelo teórico conocido como Máquina de Turing, no se disponía de un marco formal para trabajar en ciencias de la computación.
- La Máquina de Turing es un modelo matemático implementable mediante un autómata, que tiene la capacidad de implementar cualquier problema matemático, que pueda ser expresado a través de un algoritmo.

Componentes La Máquina de Turing

- Una cinta de longitud infinita dividida en celdas contiguas.

Componentes La Máquina de Turing

- Una cinta de longitud infinita dividida en celdas contiguas.
- Un cabezal que puede avanzar o retroceder la cinta en un paso discreto. (un paso = una celda), y que además puede leer o escribir en la cinta.

Componentes La Máquina de Turing

- Una cinta de longitud infinita dividida en celdas contiguas.
- Un cabezal que puede avanzar o retroceder la cinta en un paso discreto. (un paso = una celda), y que además puede leer o escribir en la cinta.
- Un registro de estado, que describe el estado en que se encuentra la máquina. (Turing lo asimilaba al estado mental de una persona cuando realiza un cálculo mental)

Componentes La Máquina de Turing

- Una cinta de longitud infinita dividida en celdas contiguas.
- Un cabezal que puede avanzar o retroceder la cinta en un paso discreto. (un paso = una celda), y que además puede leer o escribir en la cinta.
- Un registro de estado, que describe el estado en que se encuentra la máquina. (Turing lo asimilaba al estado mental de una persona cuando realiza un cálculo mental)
- Una tabla de instrucciones, que permite en función del valor leído y eventualmente de alguna acción ingresada por el usuario:

  escribir un resultado,
  avanzar la cinta,
  retroceder la cinta,
  dejar la cinta en su mismo lugar,
  y actualizar el nuevo estado en el registro de estados.

La Contribución fundamental de Alan Turing

- De éste modo, Turing proporcionó la descripción matemática de un dispositivo muy simple con capacidades de cómputo arbitrarias, capaz de probar capacidades de computabilidad tanto generales como particulares.

La Contribución fundamental de Alan Turing

- De éste modo, Turing proporcionó la descripción matemática de un dispositivo muy simple con capacidades de cómputo arbitrarias, capaz de probar capacidades de computabilidad tanto generales como particulares.
- La *Completitud Turing* de un sistema de instrucciones es la medida de su capacidad de simular una máquina de Turing.

La Contribución fundamental de Alan Turing

- De éste modo, Turing proporcionó la descripción matemática de un dispositivo muy simple con capacidades de cómputo arbitrarias, capaz de probar capacidades de computabilidad tanto generales como particulares.
- La *Completitud Turing* de un sistema de instrucciones es la medida de su capacidad de simular una máquina de Turing.
- En la actualidad los lenguajes de computación utilizados son Turing Completos siempre que no se considere que ejecutan sus programas en memoria finita.

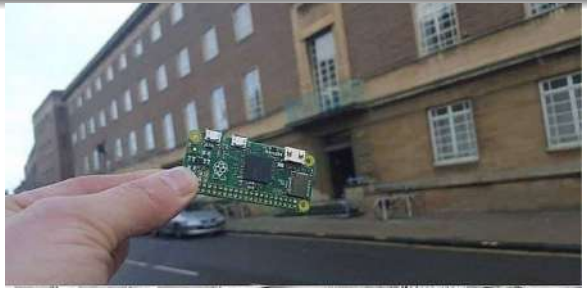
1 Pioneros

2 Evolución
- Vorágine evolutiva
- La era del Silicio
- Nace un paradigma de Arquitectura: RISC

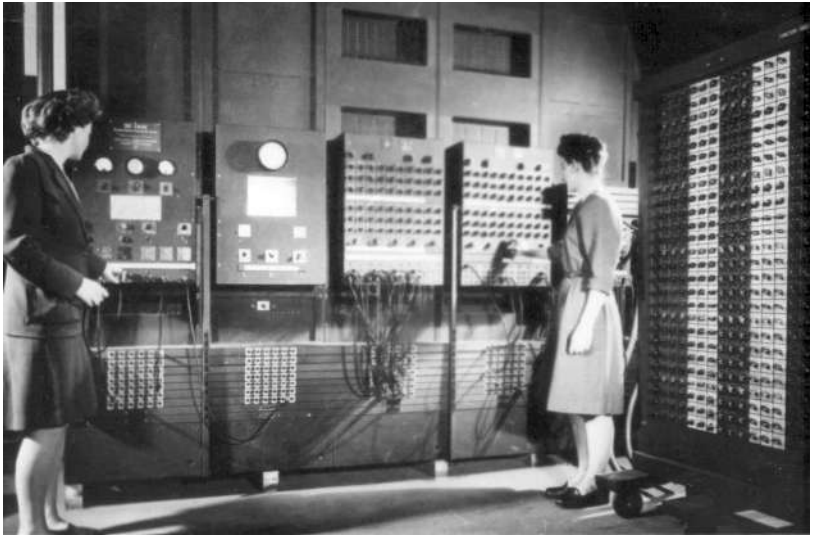
3 Arquitectura y Organización

4 Implementando la arquitectura

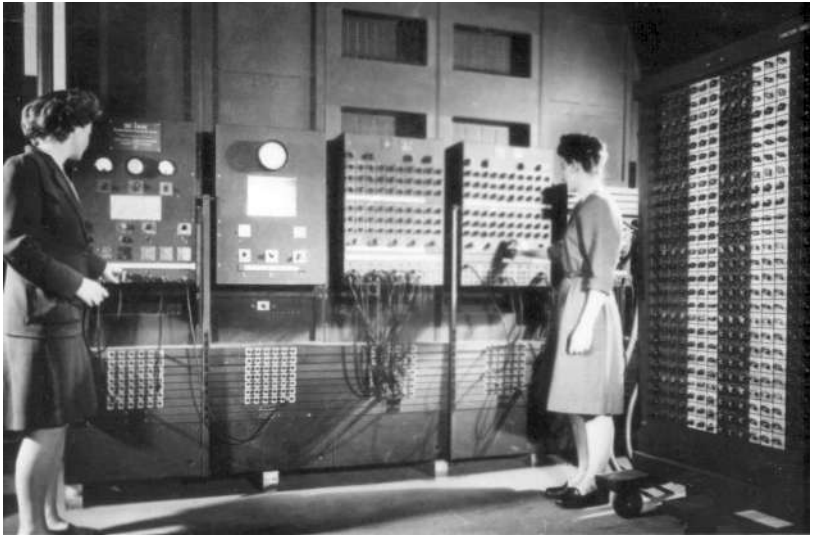
En solo algo menos de 80 años



ENIAC “primer”Computadora de Propósito general



ENIAC “primer”Computadora de Propósito general



Tal parece que la Z1 desarrollada en Alemania durante la WWII fue algo anterior.

Pero fue destruida al final de la guerra por los militares alemanes.

ENIAC está considerada la primera. No utilizaba programa almacenado, sino que se programaba mediante 6000 interruptores.

Consumo 160 KW.
Peso: 27 Toneladas.
Superficie: 167 m^2.

ENIAC BackPlane. Observar las vávulas termoiónicas

Modelo de Von Mewmann

- En 1945 se publica “First Draft of a Report on the EDVAC. John Von Newmann”.

Modelo de Von Mewmann

- En 1945 se publica “First Draft of a Report on the EDVAC. John Von Newmann”.
- Este reporte borrador describe la organización y arquitectura de una máquina de cómputo binaria y electrónica en la que se detallan las características y recursos de arquitectura de los componentes principales.

Modelo de Von Mewmann

- En 1945 se publica “First Draft of a Report on the EDVAC. John Von Newmann”.
- Este reporte borrador describe la organización y arquitectura de una máquina de cómputo binaria y electrónica en la que se detallan las características y recursos de arquitectura de los componentes principales.
- Tal vez su mérito principal sea el de haber sido la primer publicación de la arquitectura y organización de un computador basado en los postulados de Turing de una década antes.
  Obviamente la **EDVAC** (por **E**lectronic **D**iscrete **V**ariable **A**utomatic **C**omputer) era una máquina Turing completa.

Modelo de Von Mewmann

- Inaugurando el lema “publish or perish”, esta arquitectura por ser la primera bien documentada, se convirtió en sinónimo de programa almacenado (recordemos que la máquina de Turing había postulado las bases teóricas de este concepto 9 años antes).

Modelo de Von Mewmann

- Inaugurando el lema “publish or perish”, esta arquitectura por ser la primera bien documentada, se convirtió en sinónimo de programa almacenado (recordemos que la máquina de Turing había postulado las bases teóricas de este concepto 9 años antes).
- De modo que sin perjuicio de los valiosos aportes de Von Newmann, estos conceptos salieron de otra mente brillante: la de Alan Turing.

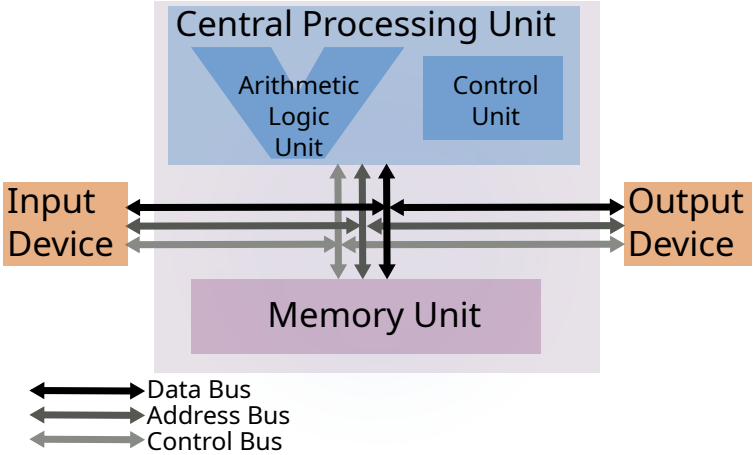
Modelo de Von Mewmann

- Inaugurando el lema “publish or perish”, esta arquitectura por ser la primera bien documentada, se convirtió en sinónimo de programa almacenado (recordemos que la máquina de Turing había postulado las bases teóricas de este concepto 9 años antes).
- De modo que sin perjuicio de los valiosos aportes de Von Newmann, estos conceptos salieron de otra mente brillante: la de Alan Turing.
- No obstante, los avances relativos de la EDVAC debidos a los trabajos de Von Newmann, y su publicación en el trabajo antes mencionado fue fundamental para la difusión del modelo de programa almacenado.

Modelo de Von Mewmann

- Inaugurando el lema “publish or perish”, esta arquitectura por ser la primera bien documentada, se convirtió en sinónimo de programa almacenado (recordemos que la máquina de Turing había postulado las bases teóricas de este concepto 9 años antes).
- De modo que sin perjuicio de los valiosos aportes de Von Newmann, estos conceptos salieron de otra mente brillante: la de Alan Turing.
- No obstante, los avances relativos de la EDVAC debidos a los trabajos de Von Newmann, y su publicación en el trabajo antes mencionado fue fundamental para la difusión del modelo de programa almacenado.
- Esta publicación fue muy influyente durante décadas en el diseño y concepción de computadores, y éste es su significativo aporte.

Modelo de Von Mewmann



Modelo simplificado de Von Newmann: Tres unidades unidas por tres buses.

Modelo de Von Mewmann

Esta organización fue dominante durante la primer y segunda generación de Computadores. Sus principales componentes se describen a continuación:

Modelo de Von Mewmann

Esta organización fue dominante durante la primer y segunda generación de Computadores. Sus principales componentes se describen a continuación:

Central Arithmetic (CA): Una Unidad de Procesamiento Aritmético Lógica y Registros.

Modelo de Von Mewmann

Esta organización fue dominante durante la primer y segunda generación de Computadores. Sus principales componentes se describen a continuación:

Central Arithmetic (CA): Una Unidad de Procesamiento Aritmético Lógica y Registros.

Central Control (CC): Implementación de la máquina de estados, con registro de instrucciones y registro Contador de Programa.

Modelo de Von Mewmann

Esta organización fue dominante durante la primer y segunda generación de Computadores. Sus principales componentes se describen a continuación:

Central Arithmetic (CA): Una Unidad de Procesamiento Aritmético Lógica y Registros.

Central Control (CC): Implementación de la máquina de estados, con registro de instrucciones y registro Contador de Programa.

Memory (M): Almacena el código y los Datos. Dos cuestiones:
(1) Esta idea ya estaba en la **ENIAC** y en otros diseños anteriores, y
(2) Con el tiempo se advirtió que este escenario limita la performance ya que no permite leer una instrucción y un dato a la vez, escenario conocido como “Von Newmann bottleneck”.

Modelo de Von Mewmann

Entrada (I): Se trata de mecanismos para que se puedan ingresar al computador comandos y datos desde el exterior.

Modelo de Von Mewmann

Entrada (I): Se trata de mecanismos para que se puedan ingresar al computador comandos y datos desde el exterior.

Salida (O): Se trata de mecanismos para que el computador pueda enviar resultados hacia el mundo exterior.

Modelo de Von Mewmann

Entrada (I): Se trata de mecanismos para que se puedan ingresar al computador comandos y datos desde el exterior.

Salida (O): Se trata de mecanismos para que el computador pueda enviar resultados hacia el mundo exterior.

Memoria Externa (R): Se trata de un medio de almacenamiento de mayor capacidad que la Unidad de Memoria (en donde solo están el código y los datos del programa en ejecución).
En la Memoria externa podemos tener una colección de programas y datos para copiar de a una vez en la Unidad de Memoria y lanzar a ejecutar. Es lo que hoy conocemos como Storage. Por entonces se trataba de Cintas perforadas por ejemplo.

Modelo de Von Meumann

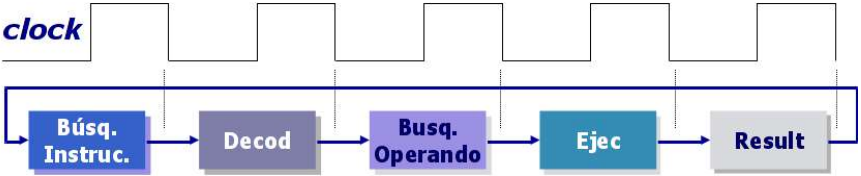
1. Modelo de programa almacenado.

Modelo de Von Meumann

1. Modelo de programa almacenado.
2. Datos y programa en el mismo (único) banco de memoria.

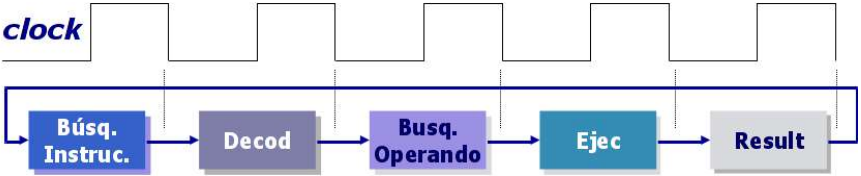
Modelo de Von Meumann

1. Modelo de programa almacenado.
2. Datos y programa en el mismo (único) banco de memoria.
3. Máquina de estados perpétua:



Modelo de Von Meumann

1. Modelo de programa almacenado.
2. Datos y programa en el mismo (único) banco de memoria.
3. Máquina de estados perpétua:



4. No está claro cual fue la primera implementación, pero Manchester Baby, IBM-SSEC, EDVAC, BINAC, entre otras fueron de las primeras (alrededor de 1948 todas ellas)

Modelo Harvard

Modelo Harvard

- Cuando Von Newmann publicó el reporte de la EDVAC, tal vez era difícil imaginar que en solo cuestión de algo mas de dos décadas, se necesitaría leer desde la memoria un código de operación y los datos necesarios para llevarla adelante **al mismo tiempo** (o al menos que todo se comporte como si así fuese).

Modelo Harvard

- Cuando Von Newmann publicó el reporte de la EDVAC, tal vez era difícil imaginar que en solo cuestión de algo mas de dos décadas, se necesitaría leer desde la memoria un código de operación y los datos necesarios para llevarla adelante **al mismo tiempo** (o al menos que todo se comporte como si así fuese).

- El postulado de Von Newmann “una única Unidad de Memoria para datos y código”, definitivamente no encaja en este nuevo paradigma.

Modelo Harvard

- Cuando Von Newmann publicó el reporte de la EDVAC, tal vez era difícil imaginar que en solo cuestión de algo mas de dos décadas, se necesitaría leer desde la memoria un código de operación y los datos necesarios para llevarla adelante **al mismo tiempo** (o al menos que todo se comporte como si así fuese).
- El postulado de Von Newmann “una única Unidad de Memoria para datos y código”, definitivamente no encaja en este nuevo paradigma.
- En este nuevo escenario, una memoria que almacene ambas entidades se transforma inevitablemente en un cuello de botella.

Modelo Harvard

- Cuando Von Newmann publicó el reporte de la EDVAC, tal vez era difícil imaginar que en solo cuestión de algo mas de dos décadas, se necesitaría leer desde la memoria un código de operación y los datos necesarios para llevarla adelante **al mismo tiempo** (o al menos que todo se comporte como si así fuese).
- El postulado de Von Newmann “una única Unidad de Memoria para datos y código”, definitivamente no encaja en este nuevo paradigma.
- En este nuevo escenario, una memoria que almacene ambas entidades se transforma inevitablemente en un cuello de botella.
- Y de este modo se rebautiza al modelo de la EDVAC como “Von Newmann bottleneck”. ¿Injusto?, tal vez, pero asi se lo conoce desde entonces

Modelo Harvard

- Recién por entonces y ante ese problema concreto, algunos diseñadores repararon en el diseño de la **Mark I**, un computador electromecánico desarrollado por IBM y la Universidad de Harvard entre 1939 y 1944.

Modelo Harvard

- Recién por entonces y ante ese problema concreto, algunos diseñadores repararon en el diseño de la **Mark I**, un computador electromecánico desarrollado por IBM y la Universidad de Harvard entre 1939 y 1944.
- Ni siquiera un computador de la primer generación, y obviamente anterior a la **EDVAC**.

Modelo Harvard

- Recién por entonces y ante ese problema concreto, algunos diseñadores repararon en el diseño de la **Mark I**, un computador electromecánico desarrollado por IBM y la Universidad de Harvard entre 1939 y 1944.
- Ni siquiera un computador de la primer generación, y obviamente anterior a la **EDVAC**.
- ¿Que podía tener de interesante esta pieza de museo compuesta de 760.000 engranajes, 800km de cables, basada en la máquina analítica de Babagge, que **trabajaba en Decimal**, y que se tomaba 0.3 a 10 segundos por cada cálculo?

Modelo Harvard

- Recién por entonces y ante ese problema concreto, algunos diseñadores repararon en el diseño de la **Mark I**, un computador electromecánico desarrollado por IBM y la Universidad de Harvard entre 1939 y 1944.
- Ni siquiera un computador de la primer generación, y obviamente anterior a la **EDVAC**.
- ¿Que podía tener de interesante esta pieza de museo compuesta de 760.000 engranajes, 800km de cables, basada en la máquina analítica de Babagge, que **trabajaba en Decimal**, y que se tomaba 0.3 a 10 segundos por cada cálculo?
- Esta reliquia, que no era electrónica ni binaria, tenía la solución al “Von Newmann bottleneck”. Así son las cosas. . .

Modelo Harvard

- Recién por entonces y ante ese problema concreto, algunos diseñadores repararon en el diseño de la **Mark I**, un computador electromecánico desarrollado por IBM y la Universidad de Harvard entre 1939 y 1944.
- Ni siquiera un computador de la primer generación, y obviamente anterior a la **EDVAC**.
- ¿Que podía tener de interesante esta pieza de museo compuesta de 760.000 engranajes, 800km de cables, basada en la máquina analítica de Babagge, que **trabajaba en Decimal**, y que se tomaba 0.3 a 10 segundos por cada cálculo?
- Esta reliquia, que no era electrónica ni binaria, tenía la solución al “Von Newmann bottleneck”. Así son las cosas. . .
- Su código ingresaba mediante una cinta de papel perforada, y los operandos mediante contadores electromecánicos. Es decir, las instrucciones y los datos ingresan por data paths diferentes.

Modelo Harvard

Conclusiones

Modelo Harvard

Conclusiones

- Un computador tiene organización Harvard **cuando las instrucciones y los datos se leen desde memorias físicamente diferentes, y por caminos de señal físicamente diferentes.**

Modelo Harvard

Conclusiones

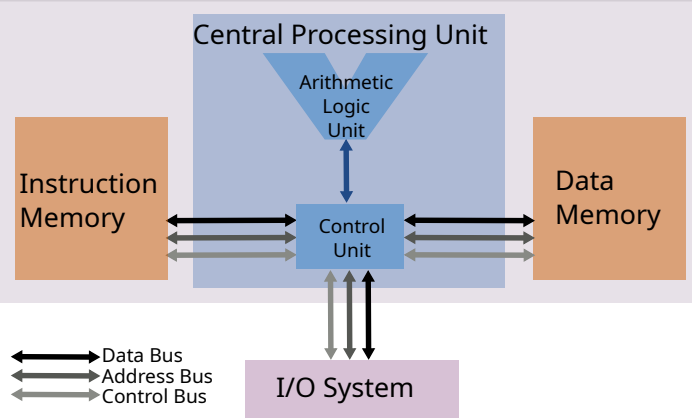
- Un computador tiene organización Harvard **cuando las instrucciones y los datos se leen desde memorias físicamente diferentes, y por caminos de señal físicamente diferentes.**
- Esto permite utilizar memorias de tecnología y organización diferente para instrucciones y para datos, y mapas de direcciones individuales para cada caso, y **caminos de señal (buses) independientes**.

Modelo Harvard

Conclusiones

- Un computador tiene organización Harvard **cuando las instrucciones y los datos se leen desde memorias físicamente diferentes, y por caminos de señal físicamente diferentes.**
- Esto permite utilizar memorias de tecnología y organización diferente para instrucciones y para datos, y mapas de direcciones individuales para cada caso, y **caminos de señal (buses) independientes**.
- O sea podemos tener dos direcciones 0x00000000 de memoria: una para instrucciones y otra para datos. La memoria de instrucciones podría estar organizada en palabras de 32 bits y la de datos en palabras de 8 bits. E igualmente todo funciona.

Modelo Harvard



Modelo Harvard. Mapas y buses de memoria diferentes para Instrucciones y Datos

Modelo Harvard

- Los microprocesadores actuales parecieran presentar al usuario una organización acorde al modelo de Von Newmann.

Modelo Harvard

- Los microprocesadores actuales parecieran presentar al usuario una organización acorde al modelo de Von Newmann.
- Sin embargo como veremos en pocas clase más, por razones de performance utilizan desde la década del 90 una tecnología denominada split cache: dividir los primeros niveles de memoria cache en una de Instrucciones y otra de datos, con sendos buses independientes para accederlas en paralelo. Esto es Harvard.

Modelo Harvard

- Los microprocesadores actuales parecieran presentar al usuario una organización acorde al modelo de Von Newmann.
- Sin embargo como veremos en pocas clase más, por razones de performance utilizan desde la década del 90 una tecnología denominada split cache: dividir los primeros niveles de memoria cache en una de Instrucciones y otra de datos, con sendos buses independientes para accederlas en paralelo. Esto es Harvard.
- Sin embargo, los niveles de memoria más externos almacenan instrucciones y datos juntos. Esto es Von Newmann.

Modelo Harvard

- Los microprocesadores actuales parecieran presentar al usuario una organización acorde al modelo de Von Newmann.
- Sin embargo como veremos en pocas clase más, por razones de performance utilizan desde la década del 90 una tecnología denominada split cache: dividir los primeros niveles de memoria cache en una de Instrucciones y otra de datos, con sendos buses independientes para accederlas en paralelo. Esto es Harvard.
- Sin embargo, los niveles de memoria más externos almacenan instrucciones y datos juntos. Esto es Von Newmann.
- El sub-sistema de memoria cache está diseñado y dimensionado de modo que se resuelvan en éste la mayoría de los accesos a memoria de parte de las CPUs.
  Un valor típico de eficiencia de un cache Nivel 1 es 80 %, es decir que la mayor parte de los acceso a memoria se resuelven en una memoria organizada de acuerdo al modelo de Harvard.

Modelo Harvard

- La única razón por la que ni siquiera el primer nivel de cache es "estrictamente Harvard" se debe que el programador "ve" un solo espacio de direccionamiento de memoria en lugar de ver uno para datos y otro para código. Esta división la realiza el sistema Cache en forma transparente.

Modelo Harvard

- La única razón por la que ni siquiera el primer nivel de cache es "estrictamente Harvard" se debe que el programador "ve" un solo espacio de direccionamiento de memoria en lugar de ver uno para datos y otro para código. Esta división la realiza el sistema Cache en forma transparente.
- Los primeros modelos de microprocesadores basados en arquitectura Harvard se dieron en la década del 80 con el advenimiento de los DSP (Procesadores de Señales Digitales), que requerían un alto nivel de paralelismo. Estos, de momento, son los únicos procesadores puramente Harvard.

Modelo Harvard

- La única razón por la que ni siquiera el primer nivel de cache es "estrictamente Harvard" se debe que el programador "ve" un solo espacio de direccionamiento de memoria en lugar de ver uno para datos y otro para código. Esta división la realiza el sistema Cache en forma transparente.
- Los primeros modelos de microprocesadores basados en arquitectura Harvard se dieron en la década del 80 con el advenimiento de los DSP (Procesadores de Señales Digitales), que requerían un alto nivel de paralelismo. Estos, de momento, son los únicos procesadores puramente Harvard.
- Los procesadores de propósito general actuales, y últimamente algún que otro microcontrolador CORTEX-M, trabajan con split cache en el Nivel 1 de su jerarquía de memoria, y juntan código y datos en los restantes niveles.

Modelo Harvard

- La única razón por la que ni siquiera el primer nivel de cache es "estrictamente Harvard" se debe que el programador "ve" un solo espacio de direccionamiento de memoria en lugar de ver uno para datos y otro para código. Esta división la realiza el sistema Cache en forma transparente.
- Los primeros modelos de microprocesadores basados en arquitectura Harvard se dieron en la década del 80 con el advenimiento de los DSP (Procesadores de Señales Digitales), que requerían un alto nivel de paralelismo. Estos, de momento, son los únicos procesadores puramente Harvard.
- Los procesadores de propósito general actuales, y últimamente algún que otro microcontrolador CORTEX-M, trabajan con split cache en el Nivel 1 de su jerarquía de memoria, y juntan código y datos en los restantes niveles.
- Todos presentan al programador un mapa de direcciones de memoria único. Razón por la cual se los llama "casi - Von Newmann".

1 Pioneros

2 Evolución
- Vorágine evolutiva
- La era del Silicio
- Nace un paradigma de Arquitectura: RISC

3 Arquitectura y Organización

4 Implementando la arquitectura

2da. Generación

2da. Generación

- Son los computadores transistorizados.

2da. Generación

- Son los computadores transistorizados.
- El transistor se implementa en 1947 en Laboratorios Bell.

2da. Generación

- Son los computadores transistorizados.
- El transistor se implementa en 1947 en Laboratorios Bell.
- Los primeros equipos aparecen durante fines de la década del 50.

2da. Generación

- Son los computadores transistorizados.
- El transistor se implementa en 1947 en Laboratorios Bell.
- Los primeros equipos aparecen durante fines de la década del 50.
- Ventajas: Menor consumo, menor espacio, y (a largo plazo) mejor desempeño.

2da. Generación

- Son los computadores transistorizados.
- El transistor se implementa en 1947 en Laboratorios Bell.
- Los primeros equipos aparecen durante fines de la década del 50.
- Ventajas: Menor consumo, menor espacio, y (a largo plazo) mejor desempeño.
- Coincide con los lenguajes de Alto Nivel.
  En 1957 se escribe el primer compilador de FORTRAN (**FOR**mula **TRAN**slator).

2da. Generación

- Son los computadores transistorizados.
- El transistor se implementa en 1947 en Laboratorios Bell.
- Los primeros equipos aparecen durante fines de la década del 50.
- Ventajas: Menor consumo, menor espacio, y (a largo plazo) mejor desempeño.
- Coincide con los lenguajes de Alto Nivel.
  En 1957 se escribe el primer compilador de FORTRAN (**FOR**mula **TRAN**slator).
- Bell Labs presenta en 1958 la TRADIC (**TRA**nsistor, **DI**gital **C**omputer)

2da. Generación

- Son los computadores transistorizados.
- El transistor se implementa en 1947 en Laboratorios Bell.
- Los primeros equipos aparecen durante fines de la década del 50.
- Ventajas: Menor consumo, menor espacio, y (a largo plazo) mejor desempeño.
- Coincide con los lenguajes de Alto Nivel.
  En 1957 se escribe el primer compilador de FORTRAN (**FOR**mula **TRAN**slator).
- Bell Labs presenta en 1958 la TRADIC (**TRA**nsistor, **DI**gital **C**omputer)
- Se consolida IBM, y nacen XEROX y DEC.

3er. Generación

3er. Generación

- Computadores basados en circuitos integrados (CI).

3er. Generación

- Computadores basados en circuitos integrados (CI).
- Jack Kirby (Texas Instruments) patenta el CI junto con una implementación en 1959. (Jacobi, Siemens AG había patentado también en 1958 solo el dispositivo).

3er. Generación

- Computadores basados en circuitos integrados (CI).
- Jack Kirby (Texas Instruments) patenta el CI junto con una implementación en 1959. (Jacobi, Siemens AG había patentado también en 1958 solo el dispositivo).
- En 1957 Robert Noyce funda Fairchild, y luego del patentamiento de Kirby le agrega una capa metálica que permite mejorar su conexionado.

3er. Generación

- Computadores basados en circuitos integrados (CI).
- Jack Kirby (Texas Instruments) patenta el CI junto con una implementación en 1959. (Jacobi, Siemens AG había patentado también en 1958 solo el dispositivo).
- En 1957 Robert Noyce funda Fairchild, y luego del patentamiento de Kirby le agrega una capa metálica que permite mejorar su conexionado.
- Igual que para la segunda generación, transcurriría un tiempo desde el descubrimiento del CI y la implementación de un computador de esta nueva generación.

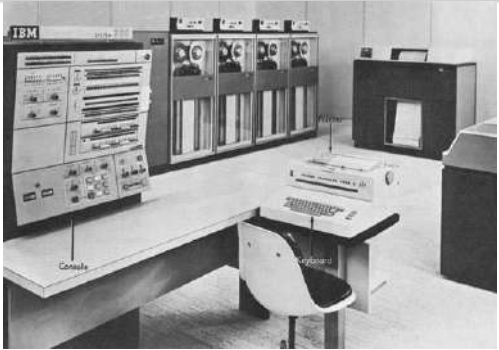
3er. Generación

- Computadores basados en circuitos integrados (CI).
- Jack Kirby (Texas Instruments) patenta el CI junto con una implementación en 1959. (Jacobi, Siemens AG había patentado también en 1958 solo el dispositivo).
- En 1957 Robert Noyce funda Fairchild, y luego del patentamiento de Kirby le agrega una capa metálica que permite mejorar su conexionado.
- Igual que para la segunda generación, transcurriría un tiempo desde el descubrimiento del CI y la implementación de un computador de esta nueva generación.
- Los primeros equipos ven la luz en los años 60.

3er. Generación

- Computadores basados en circuitos integrados (CI).
- Jack Kirby (Texas Instruments) patenta el CI junto con una implementación en 1959. (Jacobi, Siemens AG había patentado también en 1958 solo el dispositivo).
- En 1957 Robert Noyce funda Fairchild, y luego del patentamiento de Kirby le agrega una capa metálica que permite mejorar su conexionado.
- Igual que para la segunda generación, transcurriría un tiempo desde el descubrimiento del CI y la implementación de un computador de esta nueva generación.
- Los primeros equipos ven la luz en los años 60.
- El paisaje de un salón de cómputos comienza a verse muy diferente.

Minicomputadores de 1964.



- El IBM 360 cabía en una mesa.
  Los demás módulos son periféricos.
- Los PDP-8 de DEC también podían ubicarse sobre una mesa pequeña.

Otros problemas a resolver

- En los 60′, los usuarios de computadores demandaban disminuir el costo de actualización de sus equipos.

Otros problemas a resolver

- En los 60′ , los usuarios de computadores demandaban disminuir el costo de actualización de sus equipos.
- Cada computador (aun siendo del mismo fabricante) era incompatible con los otros modelos: esto es, tenía diferentes set de instrucciones, los toolchains de desarrollo de software eran diferentes, el S.O. era diferente, la E/S era diferente, y hasta el mercado era diferente para cada modelo.

Otros problemas a resolver

- En los 60′ , los usuarios de computadores demandaban disminuir el costo de actualización de sus equipos.
- Cada computador (aun siendo del mismo fabricante) era incompatible con los otros modelos: esto es, tenía diferentes set de instrucciones, los toolchains de desarrollo de software eran diferentes, el S.O. era diferente, la E/S era diferente, y hasta el mercado era diferente para cada modelo.
- Cada upgrade obligaba a reemplazar no solo el hardware sino además el sistema operativo y las aplicaciones.

Otros problemas a resolver

- En los 60′, los usuarios de computadores demandaban disminuir el costo de actualización de sus equipos.
- Cada computador (aun siendo del mismo fabricante) era incompatible con los otros modelos: esto es, tenía diferentes set de instrucciones, los toolchains de desarrollo de software eran diferentes, el S.O. era diferente, la E/S era diferente, y hasta el mercado era diferente para cada modelo.
- Cada upgrade obligaba a reemplazar no solo el hardware sino además el sistema operativo y las aplicaciones.
- Ésta dinámica dificultaba a los proveedores establecer un cash flow que permitiese incrementar inversiones en I+D.

Otros problemas a resolver

- En los 60′, los usuarios de computadores demandaban disminuir el costo de actualización de sus equipos.
- Cada computador (aun siendo del mismo fabricante) era incompatible con los otros modelos: esto es, tenía diferentes set de instrucciones, los toolchains de desarrollo de software eran diferentes, el S.O. era diferente, la E/S era diferente, y hasta el mercado era diferente para cada modelo.
- Cada upgrade obligaba a reemplazar no solo el hardware sino además el sistema operativo y las aplicaciones.
- Ésta dinámica dificultaba a los proveedores establecer un cash flow que permitiese incrementar inversiones en I+D.
- La palabra "Compatibilidad" comienza a escucharse en cada laboratorio de diseño de Procesadores y Computadores (especialmente en IBM).

Otros problemas a resolver

- El diseño de un computador se divide en dos aspectos fundamentales.

Otros problemas a resolver

- El diseño de un computador se divide en dos aspectos fundamentales.
  1. **Datapath**: Conjunto de recursos para que los números se almacenen en registros y las operaciones entre ellos se puedan ejecutar

Otros problemas a resolver

- El diseño de un computador se divide en dos aspectos fundamentales.
  1. **Datapath**: Conjunto de recursos para que los números se almacenen en registros y las operaciones entre ellos se puedan ejecutar
  2. **Control**: La lógica necesaria para que las operaciones se envíen en la secuencia correcta por el **Datapath**.

Otros problemas a resolver

- El diseño de un computador se divide en dos aspectos fundamentales.
  1. **Datapath**: Conjunto de recursos para que los números se almacenen en registros y las operaciones entre ellos se puedan ejecutar
  2. **Control**: La lógica necesaria para que las operaciones se envíen en la secuencia correcta por el **Datapath**.
- De los dos aspectos, el **Control** es la madre de todas las batallas.

Otros problemas a resolver

- El diseño de un computador se divide en dos aspectos fundamentales.
  1. **Datapath**: Conjunto de recursos para que los números se almacenen en registros y las operaciones entre ellos se puedan ejecutar
  2. **Control**: La lógica necesaria para que las operaciones se envíen en la secuencia correcta por el **Datapath**.
- De los dos aspectos, el **Control** es la madre de todas las batallas.
- Maurice Wilkes en IBM, tuvo una idea disruptiva para diseñar la lógica de control de un microprocesador: la llamó ***microprogramación*** o **microcódigo**.

Otros problemas a resolver

- El diseño de un computador se divide en dos aspectos fundamentales.
  1. **Datapath**: Conjunto de recursos para que los números se almacenen en registros y las operaciones entre ellos se puedan ejecutar
  2. **Control**: La lógica necesaria para que las operaciones se envíen en la secuencia correcta por el **Datapath**.
- De los dos aspectos, el **Control** es la madre de todas las batallas.
- Maurice Wilkes en IBM, tuvo una idea disruptiva para diseñar la lógica de control de un microprocesador: la llamó ***microprogramación*** o **microcódigo**.
- En ese momento la memoria ROM era más barata y rápida que la RAM, así que propuso implementar el **microcódigo** en una memoria ROM, en la cual cada palabra se llama microinstrucción, y cada instrucciones se descompone en una secuencia de microinstrucciones (o pequeños programas).

Otros problemas a resolver

- El diseño de un computador se divide en dos aspectos fundamentales.
  1. **Datapath**: Conjunto de recursos para que los números se almacenen en registros y las operaciones entre ellos se puedan ejecutar
  2. **Control**: La lógica necesaria para que las operaciones se envíen en la secuencia correcta por el **Datapath**.
- De los dos aspectos, el **Control** es la madre de todas las batallas.
- Maurice Wilkes en IBM, tuvo una idea disruptiva para diseñar la lógica de control de un microprocesador: la llamó ***microprogramación*** o **microcódigo**.
- En ese momento la memoria ROM era más barata y rápida que la RAM, así que propuso implementar el **microcódigo** en una memoria ROM, en la cual cada palabra se llama microinstrucción, y cada instrucciones se descompone en una secuencia de microinstrucciones (o pequeños programas).
- En abril de 1964, IBM anuncia que todos sus modelos de mainframes IBM 360 se basarán en **microcódigo**. El ancho de las microinstrucciones varió desde 50 bit a 87 bit, para **Datapaths** de 8 a 64 bit.

Microcódigo. La solución para utilizar el mínimo de memoria

El control por **microcódigo** derivó indefectiblemente en diseño del ISA con instrucciones de complejidad creciente, cargándole toda su complejidad al **microcódigo** en la CPU, y ocupando la menor cantidad de memoria posible.
Con los bloques de datos no hay mucho que hacer para ahorrar memoria.
Pero en el caso del código, si utilizamos menos instrucciones para un mismo algoritmo, se emplea menos cantidad de memoria.
Este criterio primó en todos los diseños durante décadas. Incluso cuando Intel desarrolló la arquitectura iAPx86. Junto con el compromiso de mantener **compatibilidad**, explica porque hasta el presente seguimos utilizando procesadores de instrucciones complejas.

Instrucciones complejas

1. En ese contexto se han diseñado Microprocesadores capaces de integrar código de alto nivel.

Instrucciones complejas

1. En ese contexto se han diseñado Microprocesadores capaces de integrar código de alto nivel.
2. Ejemplos extremos: `POLY (VAX computer)`

Instrucciones complejas

1. En ese contexto se han diseñado Microprocesadores capaces de integrar código de alto nivel.
2. Ejemplos extremos: `POLY (VAX computer)`

```
! To compute P(x) = C0 + C1*x + C2*x**2
! where C0 = 1.0,  C1 = .5, and C2 = .25

        POLYF    X,#2,PTABLE
        .
        .
        .
PTABLE: .FLOAT   0.25      ; C2
        .FLOAT   0.5       ; C1
        .FLOAT   1.0       ; C0
```

La 3er. Generación produjo los Minicomputadores

Minicomputadores

Costaban un orden menos que las enormes computadores transistorizados, y aumentaban en el mismo orden su potencia de cálculo.

El descenso en el costo, permite fabricar en serie al menos algunos miles de equipos, hecho que imprime impulso al desarrollo de la industria.

Este mayor poder de cómputo del hardware posibilita el desarrollo de aplicaciones más complejas, evidenciando los inconvenientes de los lenguajes de programación como Fortran, y la falta de metodologías de desarrollo.

La 3er. Generación motivó la “Programación Estructurada”

La 3er. Generación motivó la “Programación Estructurada”

- Nace el paradigma de programación estructurada.

La 3er. Generación motivó la “Programación Estructurada”

- Nace el paradigma de programación estructurada.
- En 1969 Dennis Ritchie y Ken Thompson presentan en Bell Labs un sistema operativo multitarea y multiusuario llamado UNIX, en un computador PDP-7 de Digital.

La 3er. Generación motivó la “Programación Estructurada”

- Nace el paradigma de programación estructurada.
- En 1969 Dennis Ritchie y Ken Thompson presentan en Bell Labs un sistema operativo multitarea y multiusuario llamado UNIX, en un computador PDP-7 de Digital.
- A principios de los años 70 Ritchie libera el primer compilador de lenguaje C, y documenta la sintaxis del lenguaje.

La 3er. Generación motivó la “Programación Estructurada”

- Nace el paradigma de programación estructurada.
- En 1969 Dennis Ritchie y Ken Thompson presentan en Bell Labs un sistema operativo multitarea y multiusuario llamado UNIX, en un computador PDP-7 de Digital.
- A principios de los años 70 Ritchie libera el primer compilador de lenguaje C, y documenta la sintaxis del lenguaje.
- En 1970 Ritchie y Thompson escriben la primer versión de **UNIX** casi en su totalidad en C. Primer sistema operativo portatable de la historia.

La 3er. Generación motivó la “Programación Estructurada”

- Nace el paradigma de programación estructurada.
- En 1969 Dennis Ritchie y Ken Thompson presentan en Bell Labs un sistema operativo multitarea y multiusuario llamado UNIX, en un computador PDP-7 de Digital.
- A principios de los años 70 Ritchie libera el primer compilador de lenguaje C, y documenta la sintaxis del lenguaje.
- En 1970 Ritchie y Thompson escriben la primer versión de **UNIX** casi en su totalidad en C. Primer sistema operativo portatable de la historia.
- El C se impone para escribir sistemas operativos.
  Confirma la factibilidad de desarrollar código portable y
  al ser naturalmente estructurado, se emplea también para desarrollar aplicaciones.

La 3er. Generación motivó la “Programación Estructurada”

- Nace el paradigma de programación estructurada.
- En 1969 Dennis Ritchie y Ken Thompson presentan en Bell Labs un sistema operativo multitarea y multiusuario llamado UNIX, en un computador PDP-7 de Digital.
- A principios de los años 70 Ritchie libera el primer compilador de lenguaje C, y documenta la sintaxis del lenguaje.
- En 1970 Ritchie y Thompson escriben la primer versión de **UNIX** casi en su totalidad en C. Primer sistema operativo portatable de la historia.
- El C se impone para escribir sistemas operativos.
  Confirma la factibilidad de desarrollar código portable y
  al ser naturalmente estructurado, se emplea también para desarrollar aplicaciones.
- Será durante muchos años el lenguaje de programación más utilizado.

Se consolida la Tecnología CMOS

Se consolida la Tecnología CMOS

- En los años 60 se consolida el uso de Transistores de Efecto de Campo y la tecnología MOSFET comienza a impulsar la industria de los semiconductores.

Se consolida la Tecnología CMOS

- En los años 60 se consolida el uso de Transistores de Efecto de Campo y la tecnología MOSFET comienza a impulsar la industria de los semiconductores.
- La miniaturización de los componentes de un circuito integrado es creciente y se acelera. Por eso los computadores de esta generación son mucho más compactos.

Se consolida la Tecnología CMOS

- En los años 60 se consolida el uso de Transistores de Efecto de Campo y la tecnología MOSFET comienza a impulsar la industria de los semiconductores.
- La miniaturización de los componentes de un circuito integrado es creciente y se acelera. Por eso los computadores de esta generación son mucho más compactos.
- En 1968 Robert Noyce deja Fairchild, y con Gordon Moore y Andrew Grove funda una nueva empresa. La llaman Intel. Noyce fue su primer CEO. La evolución de Intel y su contribución a la industria y a la ciencia de computación, junto con Texas, Fairchild, y otras empresas, es historia conocida.

Se consolida la Tecnología CMOS

- En los años 60 se consolida el uso de Transistores de Efecto de Campo y la tecnología MOSFET comienza a impulsar la industria de los semiconductores.
- La miniaturización de los componentes de un circuito integrado es creciente y se acelera. Por eso los computadores de esta generación son mucho más compactos.
- En 1968 Robert Noyce deja Fairchild, y con Gordon Moore y Andrew Grove funda una nueva empresa. La llaman Intel. Noyce fue su primer CEO. La evolución de Intel y su contribución a la industria y a la ciencia de computación, junto con Texas, Fairchild, y otras empresas, es historia conocida.
- Esto genera un desarrollo impresionante en el Valle de Santa Clara en California del Norte, zona que pasa a conocerse como Silicon Valley. Robert Noyce fue tan crucial en este proceso que se lo bautizó como el ‘Alcalde del Silicon Valley”.

Se consolida la Tecnología CMOS

- En los años 60 se consolida el uso de Transistores de Efecto de Campo y la tecnología MOSFET comienza a impulsar la industria de los semiconductores.
- La miniaturización de los componentes de un circuito integrado es creciente y se acelera. Por eso los computadores de esta generación son mucho más compactos.
- En 1968 Robert Noyce deja Fairchild, y con Gordon Moore y Andrew Grove funda una nueva empresa. La llaman Intel. Noyce fue su primer CEO. La evolución de Intel y su contribución a la industria y a la ciencia de computación, junto con Texas, Fairchild, y otras empresas, es historia conocida.
- Esto genera un desarrollo impresionante en el Valle de Santa Clara en California del Norte, zona que pasa a conocerse como Silicon Valley. Robert Noyce fue tan crucial en este proceso que se lo bautizó como el ‘Alcalde del Silicon Valley”.
- Comienza una vorágine tecnológica que transformará de manera sustancial la forma de vida en las próximas décadas.

Aparecen los primeros microprocesadores

Aparecen los primeros microprocesadores

- Inicialmente Intel fabricaba CI de memorias. Busicom, compañía Japonesa fabricante de Calculadoras, les pide diseñar una pequeña CPU en un CI.

Aparecen los primeros microprocesadores

- Inicialmente Intel fabricaba CI de memorias. Busicom, compañía Japonesa fabricante de Calculadoras, les pide diseñar una pequeña CPU en un CI.
- El proyecto se atasca, por dificultades para integrar los 2300 transistores.

Aparecen los primeros microprocesadores

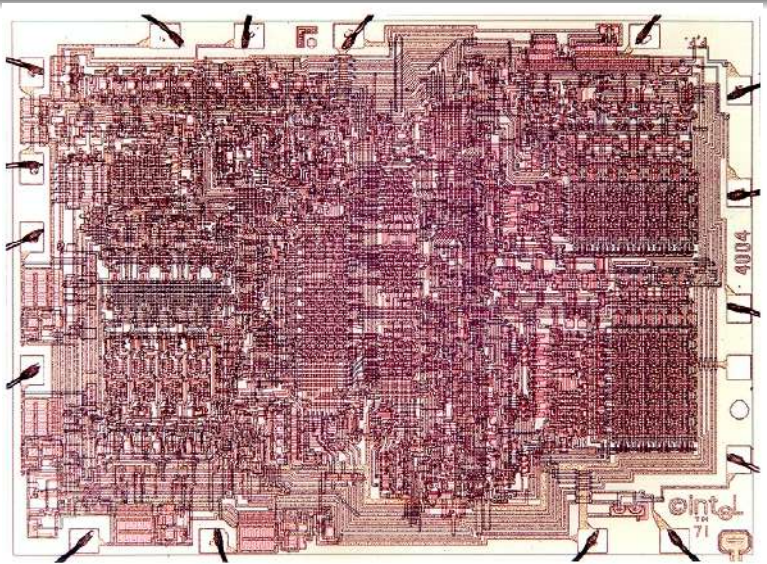
- Inicialmente Intel fabricaba CI de memorias. Busicom, compañía Japonesa fabricante de Calculadoras, les pide diseñar una pequeña CPU en un CI.
- El proyecto se atasca, por dificultades para integrar los 2300 transistores.
- Noyce contrata a Federico Faggin, de Fairchild, quien era autor de la patente de Silicon Gate Technology (SGT), tecnología crucial para lograr este objetivo.

Aparecen los primeros microprocesadores

- Inicialmente Intel fabricaba CI de memorias. Busicom, compañía Japonesa fabricante de Calculadoras, les pide diseñar una pequeña CPU en un CI.
- El proyecto se atasca, por dificultades para integrar los 2300 transistores.
- Noyce contrata a Federico Faggin, de Fairchild, quien era autor de la patente de Silicon Gate Technology (SGT), tecnología crucial para lograr este objetivo.
- En 1971 Intel presenta el CI 4004, primer Microprocesador de la historia.

  Arquitectura de 4 bit Turing Completa (16 registros de 4 bit).
  Tecnología de 15 µm (o 15 micras según la jerga de esa época).
  Clock de 740 kHz, y 12 bit para direcciones (4 KiB de direccionamiento de memoria).
  Espacios de direcciones separados para instrucciones y datos
  (Se le atribuye en forma errónea en ciertos documentos arquitectura Harvard).
  Tenía 16 terminales. Multiplexa la dirección de 12 bit con los mismos 4 bit de datos.

El 4004 de Intel



Y UNIX ya había llegado en 1969

Y UNIX ya había llegado en 1969

- La primera versión demo se desarrolló en una DEC PDP-7 íntegramente en Assembler.

Y UNIX ya había llegado en 1969

- La primera versión demo se desarrolló en una DEC PDP-7 íntegramente en Assembler.
- La figura muestra una DEC PDP-11.

Y UNIX ya había llegado en 1969

- La primera versión demo se desarrolló en una DEC PDP-7 íntegramente en Assembler.
- La figura muestra una DEC PDP-11.
- A esta máquina se portó la primer versión de UNIX escrita en C en 1970.

Y UNIX ya había llegado en 1969

- La primera versión demo se desarrolló en una DEC PDP-7 íntegramente en Assembler.
- La figura muestra una DEC PDP-11.
- A esta máquina se portó la primer versión de UNIX escrita en C en 1970.
- Contaba con 16 KiB de RAM y 512 KiB de almacenamiento. . .

Y UNIX ya había llegado en 1969

- La primera versión demo se desarrolló en una DEC PDP-7 íntegramente en Assembler.
- La figura muestra una DEC PDP-11.
- A esta máquina se portó la primer versión de UNIX escrita en C en 1970.
- Contaba con 16 KiB de RAM y 512 KiB de almacenamiento...
- No comprendo el motivo de esa sonrisa socarrona que tienen todos...

La Ley de Moore

La Ley de Moore

- El desarrollo de la tecnología de integración evolucionó de acuerdo con la predicción de Gordon Moore:

  "el número de transistores por unidad de superficie en circuitos integrados se duplica cada 2 años tendencia que continuará durante las siguientes décadas."

La Ley de Moore

- El desarrollo de la tecnología de integración evolucionó de acuerdo con la predicción de Gordon Moore:

  "el número de transistores por unidad de superficie en circuitos integrados se duplica cada 2 años tendencia que continuará durante las siguientes décadas."

- Desde 2018 aproximadamente se han alcanzado dimensiones físicas de niveles atómicos para construir un transistor MOSFET, de modo que ya no es posible reducir mucho más las dimensiones de una compuerta.

La Ley de Moore

- El desarrollo de la tecnología de integración evolucionó de acuerdo con la predicción de Gordon Moore:

  "el número de transistores por unidad de superficie en circuitos integrados se duplica cada 2 años tendencia que continuará durante las siguientes décadas."

- Desde 2018 aproximadamente se han alcanzado dimensiones físicas de niveles atómicos para construir un transistor MOSFET, de modo que ya no es posible reducir mucho más las dimensiones de una compuerta.
- Estamos presenciando el fin de esta etapa. El futuro de la industria está abierto.

La Ley de Moore

- El desarrollo de la tecnología de integración evolucionó de acuerdo con la predicción de Gordon Moore:

  "el número de transistores por unidad de superficie en circuitos integrados se duplica cada 2 años tendencia que continuará durante las siguientes décadas."

- Desde 2018 aproximadamente se han alcanzado dimensiones físicas de niveles atómicos para construir un transistor MOSFET, de modo que ya no es posible reducir mucho más las dimensiones de una compuerta.
- Estamos presenciando el fin de esta etapa. El futuro de la industria está abierto.

- Alrededor de 1980 se comienza a hablar de Circuitos Integrados VLSI (Very Large Scale Integration), en referencia a circuitos integrados que contienen cientos de miles de transistores integrados.

Se empieza a acelerar la industria

Se empieza a acelerar la industria

- Con la tecnología SGT Intel lanza los procesadores 8008, 4040 y 8080, Motorola se plantea como competidor con su procesador 6800.

Se empieza a acelerar la industria

- Con la tecnología SGT Intel lanza los procesadores 8008, 4040 y 8080, Motorola se plantea como competidor con su procesador 6800.
- Faggin plantea en Intel consolidar a la compañía como un fabricante de Microprocesadores. Noyce y fundamentalmente Gordon Moore no se deciden a abandonar el segmento de mercado de memorias.

Se empieza a acelerar la industria

- Con la tecnología SGT Intel lanza los procesadores 8008, 4040 y 8080, Motorola se plantea como competidor con su procesador 6800.
- Faggin plantea en Intel consolidar a la compañía como un fabricante de Microprocesadores. Noyce y fundamentalmente Gordon Moore no se deciden a abandonar el segmento de mercado de memorias.
- En 1974 Faggin deja Intel y funda ZILOG. En Marzo de 1976 lanza el Z80, que supera claramente al 8080 y al 8085 con el que Intel intentará contrarrestarlo, y por supuesto al 6800.

Se empieza a acelerar la industria

- Con la tecnología SGT Intel lanza los procesadores 8008, 4040 y 8080, Motorola se plantea como competidor con su procesador 6800.
- Faggin plantea en Intel consolidar a la compañía como un fabricante de Microprocesadores. Noyce y fundamentalmente Gordon Moore no se deciden a abandonar el segmento de mercado de memorias.
- En 1974 Faggin deja Intel y funda ZILOG. En Marzo de 1976 lanza el Z80, que supera claramente al 8080 y al 8085 con el que Intel intentará contrarrestarlo, y por supuesto al 6800.
- Primeros computadores personales. Altair, Commodore, Sinclair, MSX, etc. utilizaron procesadores de 8 bit, particularmente el Z80, que se transforma en el procesador más empleado.

Se empieza a acelerar la industria

- Con la tecnología SGT Intel lanza los procesadores 8008, 4040 y 8080, Motorola se plantea como competidor con su procesador 6800.
- Faggin plantea en Intel consolidar a la compañía como un fabricante de Microprocesadores. Noyce y fundamentalmente Gordon Moore no se deciden a abandonar el segmento de mercado de memorias.
- En 1974 Faggin deja Intel y funda ZILOG. En Marzo de 1976 lanza el Z80, que supera claramente al 8080 y al 8085 con el que Intel intentará contrarrestarlo, y por supuesto al 6800.
- Primeros computadores personales. Altair, Commodore, Sinclair, MSX, etc. utilizaron procesadores de 8 bit, particularmente el Z80, que se transforma en el procesador más empleado.
- No era para menos. El Z80 era compatible con el 8080 (Faggin lo conocía a la perfección, obviamente), y además lo reforzaba con las instrucciones de manejo de bits propias del 6800, combinando en un solo procesador lo mejor de los dos mundos.

El efecto Z80

El efecto Z80

- En 1975 Intel ofrecía dos procesadores: 8008 y 8080. Ambos diseños de Faggin.

El efecto Z80

- En 1975 Intel ofrecía dos procesadores: 8008 y 8080. Ambos diseños de Faggin.
- Gordon Moore impulsó el proyecto de un procesador muy superior, dando origen en ese año al proyecto 8800, que derivaría en la arquitectura iAPX432.

El efecto Z80

- En 1975 Intel ofrecía dos procesadores: 8008 y 8080. Ambos diseños de Faggin.
- Gordon Moore impulsó el proyecto de un procesador muy superior, dando origen en ese año al proyecto 8800, que derivaría en la arquitectura iAPX432.
- El diseño fue pensado como una arquitectura pura de 32 bit, con el objetivo de ser la nave insignia de Intel en las ofertas de procesadores en la década de 1980.

El efecto Z80

- En 1975 Intel ofrecía dos procesadores: 8008 y 8080. Ambos diseños de Faggin.
- Gordon Moore impulsó el proyecto de un procesador muy superior, dando origen en ese año al proyecto 8800, que derivaría en la arquitectura iAPX432.
- El diseño fue pensado como una arquitectura pura de 32 bit, con el objetivo de ser la nave insignia de Intel en las ofertas de procesadores en la década de 1980.
- Considerablemente más poderoso y complejo que el 8080, el diseño superaba ampliamente las posibilidades del proceso tecnológico existente (Al apurar su lanzamiento, que terminó lanzado en 1981, debió ser dividido en varios chips individuales.)

El efecto Z80

- En 1975 Intel ofrecía dos procesadores: 8008 y 8080. Ambos diseños de Faggin.
- Gordon Moore impulsó el proyecto de un procesador muy superior, dando origen en ese año al proyecto 8800, que derivaría en la arquitectura iAPX432.
- El diseño fue pensado como una arquitectura pura de 32 bit, con el objetivo de ser la nave insignia de Intel en las ofertas de procesadores en la década de 1980.
- Considerablemente más poderoso y complejo que el 8080, el diseño superaba ampliamente las posibilidades del proceso tecnológico existente (Al apurar su lanzamiento, que terminó lanzado en 1981, debió ser dividido en varios chips individuales.)
- El deadline para su lanzamiento chocó de frente con el Z80, que literalmente estaba barriendo del mercado a la oferta de Intel.

El efecto Z80

- En 1975 Intel ofrecía dos procesadores: 8008 y 8080. Ambos diseños de Faggin.
- Gordon Moore impulsó el proyecto de un procesador muy superior, dando origen en ese año al proyecto 8800, que derivaría en la arquitectura iAPX432.
- El diseño fue pensado como una arquitectura pura de 32 bit, con el objetivo de ser la nave insignia de Intel en las ofertas de procesadores en la década de 1980.
- Considerablemente más poderoso y complejo que el 8080, el diseño superaba ampliamente las posibilidades del proceso tecnológico existente (Al apurar su lanzamiento, que terminó lanzado en 1981, debió ser dividido en varios chips individuales.)
- El deadline para su lanzamiento chocó de frente con el Z80, que literalmente estaba barriendo del mercado a la oferta de Intel.
- Como conclusión el proyecto 8800 no podía seguir si no se neutralizaba el avance del Z80 como arquitectura dominante.

La era de la PC

La era de la PC

- Sin perjuicio de la continuidad del proyecto 8800 al que apostaba su liderazgo futuro, Intel reúne un equipo de emergencia para diseñar un procesador más acotado que el del proyecto 8800 pero superior al Z80.

La era de la PC

- Sin perjuicio de la continuidad del proyecto 8800 al que apostaba su liderazgo futuro, Intel reúne un equipo de emergencia para diseñar un procesador más acotado que el del proyecto 8800 pero superior al Z80.
- Y en tan solo 52 semanas está listo un procesador planteado como una evolución del 8080 pero extendido a una arquitectura completa de 16 bit[1].

[1] S.P. Morse, W.B. Pohlman, and B.W. Ravenel, “The Intel 8086 Microprocessor: A 16-bit Evolution of the 8080,” Computer, vol.11, no. 6, 1978, pp. 18–27)

La era de la PC

- Sin perjuicio de la continuidad del proyecto 8800 al que apostaba su liderazgo futuro, Intel reúne un equipo de emergencia para diseñar un procesador más acotado que el del proyecto 8800 pero superior al Z80.
- Y en tan solo 52 semanas está listo un procesador planteado como una evolución del 8080 pero extendido a una arquitectura completa de 16 bit[1].
- En 1978 presenta la familia iAPx86 de la mano de su nave insignia, el 8086.

[1] S.P. Morse, W.B. Pohlman, and B.W. Ravenel, “The Intel 8086 Microprocessor: A 16-bit Evolution of the 8080,” Computer, vol.11, no. 6, 1978, pp. 18–27)

La era de la PC

- Sin perjuicio de la continuidad del proyecto 8800 al que apostaba su liderazgo futuro, Intel reúne un equipo de emergencia para diseñar un procesador más acotado que el del proyecto 8800 pero superior al Z80.
- Y en tan solo 52 semanas está listo un procesador planteado como una evolución del 8080 pero extendido a una arquitectura completa de 16 bit[1].
- En 1978 presenta la familia iAPx86 de la mano de su nave insignia, el 8086.
- Se compromete a mantener compatibilidad ascendente en los modelos subsiguientes de procesadores para satisfacer esta demanda.

[1] S.P. Morse, W.B. Pohlman, and B.W. Ravenel, “The Intel 8086 Microprocessor: A 16-bit Evolution of the 8080,” Computer, vol.11, no. 6, 1978, pp. 18–27)

La era de la PC

- Sin perjuicio de la continuidad del proyecto 8800 al que apostaba su liderazgo futuro, Intel reúne un equipo de emergencia para diseñar un procesador más acotado que el del proyecto 8800 pero superior al Z80.
- Y en tan solo 52 semanas está listo un procesador planteado como una evolución del 8080 pero extendido a una arquitectura completa de 16 bit[1].
- En 1978 presenta la familia iAPx86 de la mano de su nave insignia, el 8086.
- Se compromete a mantener compatibilidad ascendente en los modelos subsiguientes de procesadores para satisfacer esta demanda.
- Esto fue uno de los motivos de la decisión de IBM de adoptar esta familia de procesadores como base para su PC, y que los Z80 estaban parcialmente financiados por Exxon, competidor directo de IBM. Sólo requirió una versión con un bus exterior de 8 bit. Y finalmente fué el 8088.

[1] S.P. Morse, W.B. Pohlman, and B.W. Ravenel, “The Intel 8086 Microprocessor: A 16-bit Evolution of the 8080,” Computer, vol.11, no. 6, 1978, pp. 18–27)

Un cisne negro

Un cisne negro

- La elección de IBM del 8086 (o mejor dicho de su primo hermano 8088 con bus externo de 8 bit) como procesador de su primer computadora personal fue un primer impulso para Intel que estaba sufriendo fuerte erosión a manos del Z80.

Un cisne negro

- La elección de IBM del 8086 (o mejor dicho de su primo hermano 8088 con bus externo de 8 bit) como procesador de su primer computadora personal fue un primer impulso para Intel que estaba sufriendo fuerte erosión a manos del Z80.
- Pero además, el forecast de venta de IBM eran 250.000 computadoras y terminó vendiendo mas de 100 millones.

Un cisne negro

- La elección de IBM del 8086 (o mejor dicho de su primo hermano 8088 con bus externo de 8 bit) como procesador de su primer computadora personal fue un primer impulso para Intel que estaba sufriendo fuerte erosión a manos del Z80.
- Pero además, el forecast de venta de IBM eran 250.000 computadoras y terminó vendiendo mas de 100 millones.
- Y así una arquitectura pensada a corto plazo (dicho no por mí sino por su autor [2]) termina revolucionando la industria y evolucionando por más de 4 décadas. Con Uds. el cisne negro.

[2] The Intel 8086 Chip and the Future of Microprocessor Design. Stephen P. Morse. Computer , 50(4):8-9, 2017

Un cisne negro

- La elección de IBM del 8086 (o mejor dicho de su primo hermano 8088 con bus externo de 8 bit) como procesador de su primer computadora personal fue un primer impulso para Intel que estaba sufriendo fuerte erosión a manos del Z80.
- Pero además, el forecast de venta de IBM eran 250.000 computadoras y terminó vendiendo mas de 100 millones.
- Y así una arquitectura pensada a corto plazo (dicho no por mí sino por su autor [2]) termina revolucionando la industria y evolucionando por más de 4 décadas. Con Uds. el cisne negro.
- En los 80 estamos en esta etapa. Aparecen Microsoft y Apple como actores nuevos de esta historia. El MS DOS se impone como Sistema Operativo y dominaría la década del 80.

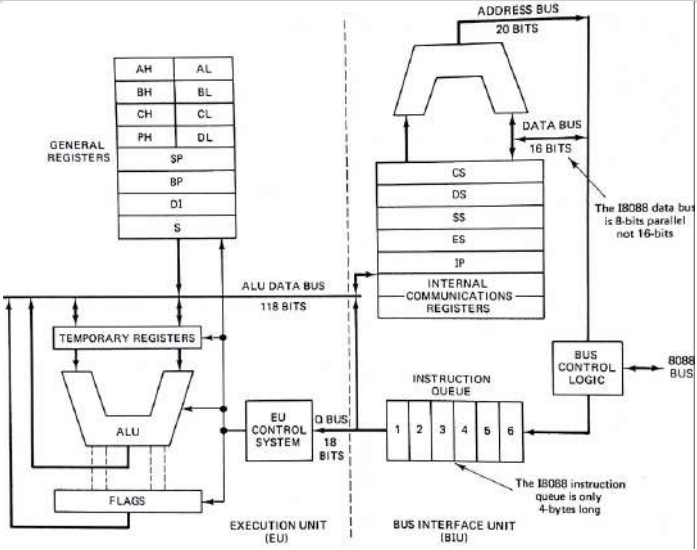
[2] The Intel 8086 Chip and the Future of Microprocessor Design. Stephen P. Morse. Computer , 50(4):8-9, 2017

Por si quedan dudas...

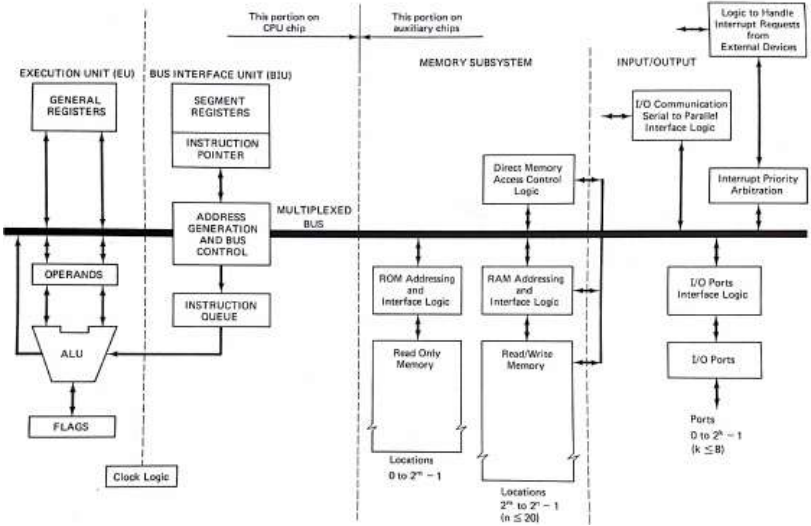
"Intel apostó su futuro a un procesador de alto desempeño, sin embargo, el diseño de dicho procesador llevaría años. Para contrarrestar a Zilog, Intel desarrolló un procesador temporal llamado 8086. Dicho procesador duraría muy poco tiempo en el mercado y no tendría sucesores, pero la historia no fue esa. El procesador de alto desempeño llegó tarde al mercado y era muy lento. De esta manera, el 8086 siguió en el mercado, y evolucionó a un procesador de 32 bits y eventualmente a 64 bits. Los nombres fueron cambiando (80186, 80286, i386, i486, Pentium), pero, por cuestiones de compatibilidad, el set de instrucciones permaneció intacto."

Stephen P. Morse, [Morse 2017]

8086 Organización



Sistema básico de 8086



Familia iAPx86

Origenes

Unos meses luego del lanzamiento del 8086, primero procesador de la familia iAPx86[3], Intel presenta a pedido de IBM una variante del mismo procesador: el 8088, idéntico en su arquitectura interna de 16 bit, pero con un bus de datos externo de 8 bit, que permitía conectarle directamente la innumerable cantidad de periféricos de 16 bit que por entonces había disponibles. Esto permitió a IBM lanzar su primer PC basada en un procesador 8088 (nunca usó el 8086).

[3] **iAPX**, se reporta como la abreviación de **i**ntel **A**dvanced **P**rocessor ar**ch**itecture, la **X** proviene de la letra griega χ (**Ji** en castellano, **Chi** en inglés)

Familia iAPx86

la primer prueba

En 1982 Intel presenta el 80286, que mejoraba la CPU 8086 en la ejecución de numerosas instrucciones e incorporó multitasking. Su arquitectura de 16 bit seguía siendo la misma, y guardaba compatibilidad a nivel binario con el 8088 (esto es, no hace falta recompilar los programas existentes). A partir de este procesador IBM diseña el modelo de PC conocido como AT.

Familia iAPx86

La consolidación en 32 bit

Familia iAPx86

La consolidación en 32 bit

- En 1984 Intel extiende la arquitectura iAPx86 a 32 bit. Lanza el 80386, y con él la arquitectura que denominará IA-32.

Familia iAPx86

La consolidación en 32 bit

- En 1984 Intel extiende la arquitectura iAPx86 a 32 bit. Lanza el 80386, y con él la arquitectura que denominará IA-32.
- La industria comienza a hablar de los procesadores x86, que en ese momento constituyen un standard. (Es el fin del sueño iAPx432)

Familia iAPx86

La consolidación en 32 bit

- En 1984 Intel extiende la arquitectura iAPx86 a 32 bit. Lanza el 80386, y con él la arquitectura que denominará IA-32.
- La industria comienza a hablar de los procesadores x86, que en ese momento constituyen un standard. (Es el fin del sueño iAPx432)
- La demanda de equipos de computo personal es astronómica superando la capacidad de Intel para abastecer al mercado.

Familia iAPx86

La consolidación en 32 bit

- En 1984 Intel extiende la arquitectura iAPx86 a 32 bit. Lanza el 80386, y con él la arquitectura que denominará IA-32.
- La industria comienza a hablar de los procesadores x86, que en ese momento constituyen un standard. (Es el fin del sueño iAPx432)
- La demanda de equipos de computo personal es astronómica superando la capacidad de Intel para abastecer al mercado.
- Intel habilita a terceros la producción de procesadores IA-32 bajo licencia, para mantener el impulso de expansión de IA-32.

Familia iAPx86

La consolidación en 32 bit

- En 1984 Intel extiende la arquitectura iAPx86 a 32 bit. Lanza el 80386, y con él la arquitectura que denominará IA-32.
- La industria comienza a hablar de los procesadores x86, que en ese momento constituyen un standard. (Es el fin del sueño iAPx432)
- La demanda de equipos de computo personal es astronómica superando la capacidad de Intel para abastecer al mercado.
- Intel habilita a terceros la producción de procesadores IA-32 bajo licencia, para mantener el impulso de expansión de IA-32.
- Nace AMD.

Familia iAPx86

La consolidación en 32 bit

- En 1984 Intel extiende la arquitectura iAPx86 a 32 bit. Lanza el 80386, y con él la arquitectura que denominará IA-32.
- La industria comienza a hablar de los procesadores x86, que en ese momento constituyen un standard. (Es el fin del sueño iAPx432)
- La demanda de equipos de computo personal es astronómica superando la capacidad de Intel para abastecer al mercado.
- Intel habilita a terceros la producción de procesadores IA-32 bajo licencia, para mantener el impulso de expansión de IA-32.
- Nace AMD.
- En los '90 Intel presenta el procesador Pentium y deja de habilitar a terceros su producción bajo licencia. Nace el lema "Intel Inside".

Aquel proyecto de 52 semanas lleva reinando más de dos décadas!

EPIC

Aquel proyecto de 52 semanas lleva reinando más de dos décadas!

EPIC

- A fines de los ’90 Intel y Hewlett Packard desarrollan una arquitectura de 64 bit.

Aquel proyecto de 52 semanas lleva reinando más de dos décadas!

EPIC

- A fines de los '90 Intel y Hewlett Packard desarrollan una arquitectura de 64 bit.
- Se basó en un concepto surgido como evolución del modelo **VLIW** (**V**ery **L**arge **I**nstruction **W**ord)

Aquel proyecto de 52 semanas lleva reinando más de dos décadas!

EPIC

- A fines de los '90 Intel y Hewlett Packard desarrollan una arquitectura de 64 bit.
- Se basó en un concepto surgido como evolución del modelo **VLIW** (**V**ery **L**arge **I**nstruction **W**ord)
- **E**xplicitly **P**arallell **I**nstruction **C**omputer (**EPIC**).

Aquel proyecto de 52 semanas lleva reinando más de dos décadas!

EPIC

- A fines de los '90 Intel y Hewlett Packard desarrollan una arquitectura de 64 bit.
- Se basó en un concepto surgido como evolución del modelo **VLIW** (**V**ery **L**arge **I**nstruction **W**ord)
- **E**xplicitly **P**arallell **I**nstruction **C**omputer (**EPIC**).
- El paralelismo de instrucciones lo resuelve el compilador y el hardware se dedica a proveer fuerza bruta de ejecución.
  Novedoso pero nunca explorado comercialmente.

Aquel proyecto de 52 semanas lleva reinando más de dos décadas!

EPIC

- A fines de los '90 Intel y Hewlett Packard desarrollan una arquitectura de 64 bit.
- Se basó en un concepto surgido como evolución del modelo **VLIW** (**V**ery **L**arge **I**nstruction **W**ord)
- **E**xplicitly **P**arallell **I**nstruction **C**omputer (**EPIC**).
- El paralelismo de instrucciones lo resuelve el compilador y el hardware se dedica a proveer fuerza bruta de ejecución.
  Novedoso pero nunca explorado comercialmente.
- Se presenta bajo el paraguas IA-64.

Aquel proyecto de 52 semanas lleva reinando más de dos décadas!

EPIC

- A fines de los '90 Intel y Hewlett Packard desarrollan una arquitectura de 64 bit.
- Se basó en un concepto surgido como evolución del modelo **VLIW** (**V**ery **L**arge **I**nstruction **W**ord)
- **E**xplicitly **P**arallell **I**nstruction **C**omputer (**EPIC**).
- El paralelismo de instrucciones lo resuelve el compilador y el hardware se dedica a proveer fuerza bruta de ejecución.
  Novedoso pero nunca explorado comercialmente.
- Se presenta bajo el paraguas IA-64.
- El procesador Itanium es el primer producto.

Aquel proyecto de 52 semanas lleva reinando más de dos décadas!

EPIC

- A fines de los '90 Intel y Hewlett Packard desarrollan una arquitectura de 64 bit.
- Se basó en un concepto surgido como evolución del modelo **VLIW** (**V**ery **L**arge **I**nstruction **W**ord)
- **E**xplicitly **P**arallell **I**nstruction **C**omputer (**EPIC**).
- El paralelismo de instrucciones lo resuelve el compilador y el hardware se dedica a proveer fuerza bruta de ejecución.
  Novedoso pero nunca explorado comercialmente.
- Se presenta bajo el paraguas IA-64.
- El procesador Itanium es el primer producto.
- No podía ejecutar código x86, ni se comercializaba junto con el compilador.

¿Y que pasó con EPIC?

¿Y que pasó con EPIC?

“EPIC Failure”[4]

[4] *“John Hennessy and David Patterson Deliver Turing Lecture at ISCA 2018”*

AMD toma su camino

Extiende IA-32 a 64 bit

AMD toma su camino

Extiende IA-32 a 64 bit

- Mientras Intel se embarcaba en el *"itanic"* en busca de una arquitectura de 64 bit, AMD toma IA-32 y la extiende a 64 bit.

AMD toma su camino

Extiende IA-32 a 64 bit

- Mientras Intel se embarcaba en el *"itanic"* en busca de una arquitectura de 64 bit, AMD toma IA-32 y la extiende a 64 bit.
- Denomina a esta arquitectura AMD-64.

AMD toma su camino

Extiende IA-32 a 64 bit

- Mientras Intel se embarcaba en el *"itanic"* en busca de una arquitectura de 64 bit, AMD toma IA-32 y la extiende a 64 bit.
- Denomina a esta arquitectura AMD-64.
- Y las protege con una patente.

AMD toma su camino

Extiende IA-32 a 64 bit

- Mientras Intel se embarcaba en el *"itanic"* en busca de una arquitectura de 64 bit, AMD toma IA-32 y la extiende a 64 bit.
- Denomina a esta arquitectura AMD-64.
- Y las protege con una patente.
- Nace el procesador Opteron, muy competitivo para Itanium tanto en costo como en compatibilidad con IA-32

AMD toma su camino

Extiende IA-32 a 64 bit

- Mientras Intel se embarcaba en el *"itanic"* en busca de una arquitectura de 64 bit, AMD toma IA-32 y la extiende a 64 bit.
- Denomina a esta arquitectura AMD-64.
- Y las protege con una patente.
- Nace el procesador Opteron, muy competitivo para Itanium tanto en costo como en compatibilidad con IA-32
- AMD-64 tiene un submodo llamado compatibilidad que permite ejecutar código IA-32 nativo sin recompilar.

1 Pioneros

2 Evolución
- Vorágine evolutiva
- La era del Silicio
- Nace un paradigma de Arquitectura: RISC

3 Arquitectura y Organización

4 Implementando la arquitectura

Orígenes de RISC

Orígenes de RISC

- Antes de las Personal Computers, el mercado era dominado por los denominados mainframes.

Orígenes de RISC

- Antes de las Personal Computers, el mercado era dominado por los denominados mainframes.
- Los players eran IBM, Digital Electronic Corporation (DEC), Amdahl, entre otros. Sus computadores hoy son piezas de museo.

Orígenes de RISC

- Antes de las Personal Computers, el mercado era dominado por los denominados mainframes.
- Los players eran IBM, Digital Electronic Corporation (DEC), Amdahl, entre otros. Sus computadores hoy son piezas de museo.
- Sobre los años 80 aparecen los primeros microprocesadores de 16 bits con ciertas capacidades para hacer frente a una nueva generación de equipos de cómputo:

Orígenes de RISC

- Antes de las Personal Computers, el mercado era dominado por los denominados mainframes.
- Los players eran IBM, Digital Electronic Corporation (DEC), Amdahl, entre otros. Sus computadores hoy son piezas de museo.
- Sobre los años 80 aparecen los primeros microprocesadores de 16 bits con ciertas capacidades para hacer frente a una nueva generación de equipos de cómputo:
  - Intel evoluciona el 8086/88 al 80286, y ensaya el iAPx432 citado como micromainframe processor, especialmente diseñado para ejecutar lenguajes de alto nivel como Ada.

Orígenes de RISC

- Antes de las Personal Computers, el mercado era dominado por los denominados mainframes.
- Los players eran IBM, Digital Electronic Corporation (DEC), Amdahl, entre otros. Sus computadores hoy son piezas de museo.
- Sobre los años 80 aparecen los primeros microprocesadores de 16 bits con ciertas capacidades para hacer frente a una nueva generación de equipos de cómputo:
  - Intel evoluciona el 8086/88 al 80286, y ensaya el iAPx432 citado como micromainframe processor, especialmente diseñado para ejecutar lenguajes de alto nivel como Ada.
  - Motorola evoluciona el 68000 (citado como m68k)

Orígenes de RISC

- Antes de las Personal Computers, el mercado era dominado por los denominados mainframes.
- Los players eran IBM, Digital Electronic Corporation (DEC), Amdahl, entre otros. Sus computadores hoy son piezas de museo.
- Sobre los años 80 aparecen los primeros microprocesadores de 16 bits con ciertas capacidades para hacer frente a una nueva generación de equipos de cómputo:
  - Intel evoluciona el 8086/88 al 80286, y ensaya el iAPx432 citado como micromainframe processor, especialmente diseñado para ejecutar lenguajes de alto nivel como Ada.
  - Motorola evoluciona el 68000 (citado como m68k)
- Vimos que tienen todos un punto en común:

Set de instrucciones de complejidad creciente

Orígenes de RISC

Líneas de Investigación

Durante los años ’60 y ’70, IBM, Control Data Corporation (CDC), y Data General (DG), incursionaron líneas de investigación tendientes a implementar procesadores con pocas instrucciones simples.
En los ’80 David Patterson[1] [2] (Universidad de Berkeley), y John Hennesy[3] (Universidad de Stanford) publicaron los primeros trabajos con resultados concretos, que presentaban una arquitectura contrapuesta con la de los Computadores que en ese momento dominaban la industria.

La denominaron **RISC** (**R**educed **I**nstruction **S**et **C**omputer). Y esto motivó que a los procesadores diseñados hasta entonces se los etiquetar como **CISC** (Por **C**omplex **I**nstrucion **S**et **C**omputer)

[1] David A. Patterson, Carlo H. Sequin. RISC I: A Reduced Instruction Set VLSI Computer

[2] David A. Patterson, David R. Ditzel. The case for the reduced instruction set computer

[3] Hennessy, J.L., Jouppi, N., Baskett, F., Gill, J. MIPS: A VLSI Processor Architecture. In Proceedings CMU Conference on VLSI Systems and Computations. Computer Science Press, October 1981

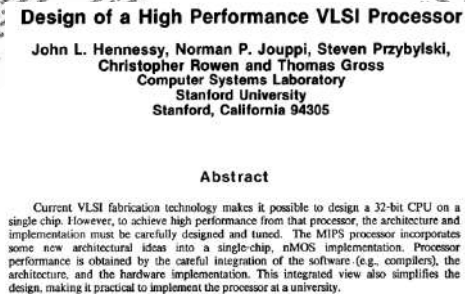
¿Porque RISC?

RISC I: A REDUCED INSTRUCTION SET VLSI COMPUTER

DAVID A. PATTERSON and CARLO H. SEQUIN

Computer Science Division
University of California
Berkeley, California

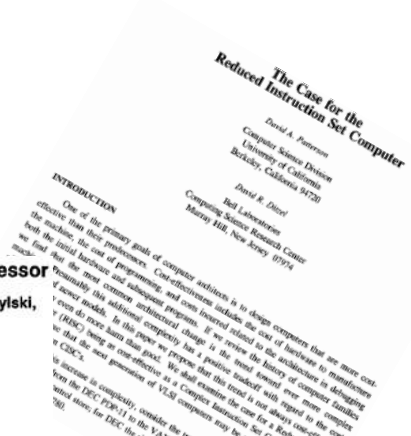
ABSTRACT


Design of a High Performance VLSI Processor

John L. Hennessy, Norman P. Jouppi, Steven Przybylski, Christopher Rowen and Thomas Gross
Computer Systems Laboratory
Stanford University
Stanford, California 94305

Abstract

Current VLSI fabrication technology makes it possible to design a 32-bit CPU on a single chip. However, to achieve high performance from that processor, the architecture and implementation must be carefully designed and tuned. The MIPS processor incorporates some new architectural ideas into a single-chip, nMOS implementation. Processor performance is obtained by the careful integration of the software (e.g., compilers), the architecture, and the hardware implementation. This integrated view also simplifies the design, making it practical to implement the processor at a university.


The Case for the Reduced Instruction Set Computer

David A. Patterson
Computer Science Division
University of California
Berkeley, California 94720

David R. Ditzel
Bell Laboratories
Computing Science Research Center
Murray Hill, New Jersey 07974

INTRODUCTION

Los Mandamientos RISC

Los Mandamientos RISC

1. Se dispone de un juego de registros numeroso, todos de propósito general.

Los Mandamientos RISC

1. Se dispone de un juego de registros numeroso, todos de propósito general.
2. Las instrucciones se ejecutan en un solo ciclo de clock.

Los Mandamientos RISC

1. Se dispone de un juego de registros numeroso, todos de propósito general.
2. Las instrucciones se ejecutan en un solo ciclo de clock.
3. Las instrucciones derivan en códigos de operación de igual formato y tamaño.

Los Mandamientos RISC

1. Se dispone de un juego de registros numeroso, todos de propósito general.
2. Las instrucciones se ejecutan en un solo ciclo de clock.
3. Las instrucciones derivan en códigos de operación de igual formato y tamaño.
4. Las instrucciones deben ser sencillas de decodificar. Los números de registros se deben tener la misma ubicación en los códigos de la instrucción y deben requerir la misma cantidad de bits para su decodificación.

Los Mandamientos RISC

1. Se dispone de un juego de registros numeroso, todos de propósito general.
2. Las instrucciones se ejecutan en un solo ciclo de clock.
3. Las instrucciones derivan en códigos de operación de igual formato y tamaño.
4. Las instrucciones deben ser sencillas de decodificar. Los números de registros se deben tener la misma ubicación en los códigos de la instrucción y deben requerir la misma cantidad de bits para su decodificación.
5. No se utiliza microcódigo para decodificar instrucciones (no hay instrucciones complejas, como DIV o MUL).

Los Mandamientos RISC

1. Se dispone de un juego de registros numeroso, todos de propósito general.
2. Las instrucciones se ejecutan en un solo ciclo de clock.
3. Las instrucciones derivan en códigos de operación de igual formato y tamaño.
4. Las instrucciones deben ser sencillas de decodificar. Los números de registros se deben tener la misma ubicación en los códigos de la instrucción y deben requerir la misma cantidad de bits para su decodificación.
5. No se utiliza microcódigo para decodificar instrucciones (no hay instrucciones complejas, como DIV o MUL).
6. Los datos en memoria se acceden mediante instrucciones simples de transferencia: LOAD y STORE.

Corolario

Corolario

1. La verificación de una máquina RISC es mucho más simple de realizar, y también es menor la probabilidad de liberar al mercado un procesador con defectos de lógica (Recordemos el bug de división del Pentium@90Mhz).

Corolario

1. La verificación de una máquina RISC es mucho más simple de realizar, y también es menor la probabilidad de liberar al mercado un procesador con defectos de lógica (Recordemos el bug de división del Pentium@90Mhz).
2. Es esperable que el tamaño de los programas sea mayor, debido a que lo que los procesadores CISC resuelven con microcódigo, los RISC lo resuelven en el software. Una multiplicación se resuelve siempre por acumulación de sumas.

$$A * B = \sum_{i=1}^{A} B$$

Corolario

1. La verificación de una máquina RISC es mucho más simple de realizar, y también es menor la probabilidad de liberar al mercado un procesador con defectos de lógica (Recordemos el bug de división del Pentium@90Mhz).
2. Es esperable que el tamaño de los programas sea mayor, debido a que lo que los procesadores CISC resuelven con microcódigo, los RISC lo resuelven en el software. Una multiplicación se resuelve siempre por acumulación de sumas.

$$A * B = \sum_{i=1}^{A} B$$

   - En microcódigo se implementa la instrucción MUL.

Corolario

1. La verificación de una máquina RISC es mucho más simple de realizar, y también es menor la probabilidad de liberar al mercado un procesador con defectos de lógica (Recordemos el bug de división del Pentium@90Mhz).

2. Es esperable que el tamaño de los programas sea mayor, debido a que lo que los procesadores CISC resuelven con microcódigo, los RISC lo resuelven en el software. Una multiplicación se resuelve siempre por acumulación de sumas.

$$A * B = \sum_{i=1}^{A} B$$

- En microcódigo se implementa la instrucción MUL.
- Por software se implementa con una subrutina.

Listado 2

```
; R1 = R1 * R2
otro:   add  R1,R1,R1
        sub  R2,R2,#1
        bne  otro
```

Sentido de esta reseña

- Durante la relativamente breve historia de esta industria, la memoria fue desde el inicio, y durante muchas décadas el recurso mas escaso (en rigor sigue vigente la máxima de Von Neumann *"la cantidad de memoria siempre será insuficiente"*).

Sentido de esta reseña

- Durante la relativamente breve historia de esta industria, la memoria fue desde el inicio, y durante muchas décadas el recurso mas escaso (en rigor sigue vigente la máxima de Von Neumann *"la cantidad de memoria siempre será insuficiente"*).
- Como hemos visto, en los albores esta escasez fue dramática. 16 KiB de RAM (8 para el sistema operativo y 8 para las aplicaciones) en 1969,. . . huelgan las palabras.

Sentido de esta reseña

- Durante la relativamente breve historia de esta industria, la memoria fue desde el inicio, y durante muchas décadas el recurso mas escaso (en rigor sigue vigente la máxima de Von Neumann *"la cantidad de memoria siempre será insuficiente"*).
- Como hemos visto, en los albores esta escasez fue dramática. 16 KiB de RAM (8 para el sistema operativo y 8 para las aplicaciones) en 1969,. . . huelgan las palabras.
- Actualmente, la memoria ha dejado de ser el principal problema ya que la tecnología de integración ha dado respuestas.

Sentido de esta reseña

- Durante la relativamente breve historia de esta industria, la memoria fue desde el inicio, y durante muchas décadas el recurso mas escaso (en rigor sigue vigente la máxima de Von Neumann *"la cantidad de memoria siempre será insuficiente"*).
- Como hemos visto, en los albores esta escasez fue dramática. 16 KiB de RAM (8 para el sistema operativo y 8 para las aplicaciones) en 1969,. . . huelgan las palabras.
- Actualmente, la memoria ha dejado de ser el principal problema ya que la tecnología de integración ha dado respuestas.
- La preocupación central en estos días es sin lugar a dudas el consumo de energía eléctrica. Basta leer las publicaciones científicas. Un alto porcentaje de los trabajos científicos propone mejoras para lograr CI que consuman menor cantidad de energía para la misma tarea (es decir menor trabajo).

Sentido de esta reseña

- Durante la relativamente breve historia de esta industria, la memoria fue desde el inicio, y durante muchas décadas el recurso mas escaso (en rigor sigue vigente la máxima de Von Neumann *"la cantidad de memoria siempre será insuficiente"*).
- Como hemos visto, en los albores esta escasez fue dramática. 16 KiB de RAM (8 para el sistema operativo y 8 para las aplicaciones) en 1969,. . . huelgan las palabras.
- Actualmente, la memoria ha dejado de ser el principal problema ya que la tecnología de integración ha dado respuestas.
- La preocupación central en estos días es sin lugar a dudas el consumo de energía eléctrica. Basta leer las publicaciones científicas. Un alto porcentaje de los trabajos científicos propone mejoras para lograr CI que consuman menor cantidad de energía para la misma tarea (es decir menor trabajo).
- En algún tiempo más esta situación estará superada. Y habrá nuevas restricciones.

Sentido de esta reseña

- Durante la relativamente breve historia de esta industria, la memoria fue desde el inicio, y durante muchas décadas el recurso mas escaso (en rigor sigue vigente la máxima de Von Neumann *"la cantidad de memoria siempre será insuficiente"*).
- Como hemos visto, en los albores esta escasez fue dramática. 16 KiB de RAM (8 para el sistema operativo y 8 para las aplicaciones) en 1969,. . . huelgan las palabras.
- Actualmente, la memoria ha dejado de ser el principal problema ya que la tecnología de integración ha dado respuestas.
- La preocupación central en estos días es sin lugar a dudas el consumo de energía eléctrica. Basta leer las publicaciones científicas. Un alto porcentaje de los trabajos científicos propone mejoras para lograr CI que consuman menor cantidad de energía para la misma tarea (es decir menor trabajo).
- En algún tiempo más esta situación estará superada. Y habrá nuevas restricciones.
- En cada época los arquitectos de computadores lidiaron con restricciones y sus decisiones siempre son producto de éstas.

¿¿¿¿¿¿¿ 9c23744 (minucias)

1 Pioneros

2 Evolución

3 Arquitectura y Organización

- Introducción

4 Implementando la arquitectura

Arquitectura vs Microarquitectura

- **Arquitectura**
  Es el conjunto de recursos accesibles para el programador, que por lo general se mantienen a lo largo de los diferentes modelos de procesadores de esa arquitectura (puede evolucionar pero la tendencia es mantener compatibilidad hacia modelos anteriores).

Arquitectura vs Microarquitectura

- **Arquitectura**
  Es el conjunto de recursos accesibles para el programador, que por lo general se mantienen a lo largo de los diferentes modelos de procesadores de esa arquitectura (puede evolucionar pero la tendencia es mantener compatibilidad hacia modelos anteriores).
  - Registros

Arquitectura vs Microarquitectura

- **Arquitectura**
  Es el conjunto de recursos accesibles para el programador, que por lo general se mantienen a lo largo de los diferentes modelos de procesadores de esa arquitectura (puede evolucionar pero la tendencia es mantener compatibilidad hacia modelos anteriores).
  - Registros
  - Set de instrucciones

Arquitectura vs Microarquitectura

- **Arquitectura**
  Es el conjunto de recursos accesibles para el programador, que por lo general se mantienen a lo largo de los diferentes modelos de procesadores de esa arquitectura (puede evolucionar pero la tendencia es mantener compatibilidad hacia modelos anteriores).
  - Registros
  - Set de instrucciones
  - Estructuras de memoria relacionadas (descriptores de página p. ej.)

Arquitectura vs Microarquitectura

- **Arquitectura**
  Es el conjunto de recursos accesibles para el programador, que por lo general se mantienen a lo largo de los diferentes modelos de procesadores de esa arquitectura (puede evolucionar pero la tendencia es mantener compatibilidad hacia modelos anteriores).
  - Registros
  - Set de instrucciones
  - Estructuras de memoria relacionadas (descriptores de página p. ej.)
- **Micro Arquitectura**
  Es la implementación de la arquitectura. El diseño lógico detrás del set de registros y del modelo de programación. Puede ser muy simple o sumamente robusta y poderosa. Cambia de un modelo a otro dentro de una misma familia.

Definición de la Arquitectura de un Computador

- Se trata de definir los aspectos más relevantes en la arquitectura de un computador procurando maximizar el rendimiento del procesador, pero sin dejar de satisfacer otras limitaciones impuestas por los usuarios, como un costo accesible, o un consumo de energía moderado, entre otros.

Definición de la Arquitectura de un Computador

- Se trata de definir los aspectos más relevantes en la arquitectura de un computador procurando maximizar el rendimiento del procesador, pero sin dejar de satisfacer otras limitaciones impuestas por los usuarios, como un costo accesible, o un consumo de energía moderado, entre otros.
- Requiere diseñar el set de instrucciones, el modo en que se manejará la memoria y sus modos de direccionamiento, los restantes bloques funcionales que componen el CPU, como el File Register, el DataPath y el diseño de la lógica de control.

Definición de la Arquitectura de un Computador

- Se trata de definir los aspectos más relevantes en la arquitectura de un computador procurando maximizar el rendimiento del procesador, pero sin dejar de satisfacer otras limitaciones impuestas por los usuarios, como un costo accesible, o un consumo de energía moderado, entre otros.
- Requiere diseñar el set de instrucciones, el modo en que se manejará la memoria y sus modos de direccionamiento, los restantes bloques funcionales que componen el CPU, como el File Register, el DataPath y el diseño de la lógica de control.
- La implementación comprende el diseño del circuito integrado, su encapsulado, montaje, alimentación y refrigeración.

Definción de la ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

- **Clases de ISA.** ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.

Definción de la ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

- **Clases de ISA.** ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.
- **Direccionamiento de Memoria.** Alineación obligatoria de datos vs. administración de a bytes.

Definción de la ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

- **Clases de ISA.** ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.
- **Direccionamiento de Memoria.** Alineación obligatoria de datos vs. administración de a bytes.
- **Modos de Direccionamiento.** Como se especifican los operandos.

Definción de la ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

- **Clases de ISA.** ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.
- **Direccionamiento de Memoria.** Alineación obligatoria de datos vs. administración de a bytes.
- **Modos de Direccionamiento.** Como se especifican los operandos.
- **Tipos y tamaños de operandos.** Enteros, Punto Flotante, Punto Fijo.Diferentes tamaños y precisiones.

Definción de la ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

- **Clases de ISA.** ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.
- **Direccionamiento de Memoria.** Alineación obligatoria de datos vs. administración de a bytes.
- **Modos de Direccionamiento.** Como se especifican los operandos.
- **Tipos y tamaños de operandos.** Enteros, Punto Flotante, Punto Fijo.Diferentes tamaños y precisiones.
- **Operaciones**. Pocas Simples (RISC) o Muchas Complejas (CISC).

Definción de la ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

- **Clases de ISA.** ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.
- **Direccionamiento de Memoria.** Alineación obligatoria de datos vs. administración de a bytes.
- **Modos de Direccionamiento.** Como se especifican los operandos.
- **Tipos y tamaños de operandos.** Enteros, Punto Flotante, Punto Fijo.Diferentes tamaños y precisiones.
- **Operaciones**. Pocas Simples (RISC) o Muchas Complejas (CISC).
- **Instrucciones de control de flujo** Saltos condicionales, calls.

Definición de la ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

- **Clases de ISA.** ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.
- **Direccionamiento de Memoria.** Alineación obligatoria de datos vs. administración de a bytes.
- **Modos de Direccionamiento.** Como se especifican los operandos.
- **Tipos y tamaños de operandos.** Enteros, Punto Flotante, Punto Fijo.Diferentes tamaños y precisiones.
- **Operaciones**. Pocas Simples (RISC) o Muchas Complejas (CISC).
- **Instrucciones de control de flujo** Saltos condicionales, calls.
- **Longitud del código** Instrucciones de tamaño fijo vs. variable.

Microarquitectura = Organización + Hardware

Organización

Se refiere a los detalles de implementación de la ISA. Es decir:

Microarquitectura = Organización + Hardware

Organización

Se refiere a los detalles de implementación de la ISA. Es decir:

- Organización e interconexión de la memoria.

Microarquitectura = Organización + Hardware

Organización

Se refiere a los detalles de implementación de la ISA. Es decir:

- Organización e interconexión de la memoria.
- Diseño de los diferentes bloques de la CPU y para implementar el set de instrucciones.

Microarquitectura = Organización + Hardware

Organización

Se refiere a los detalles de implementación de la ISA. Es decir:

- Organización e interconexión de la memoria.
- Diseño de los diferentes bloques de la CPU y para implementar el set de instrucciones.
- Implementación de paralelismo a nivel de Instrucciones, y/o de datos.

Microarquitectura = Organización + Hardware

Organización

Se refiere a los detalles de implementación de la ISA. Es decir:

- Organización e interconexión de la memoria.
- Diseño de los diferentes bloques de la CPU y para implementar el set de instrucciones.
- Implementación de paralelismo a nivel de Instrucciones, y/o de datos.
- Podemos así encontrar procesadores que poseen la misma ISA pero una organización muy diferente.

Microarquitectura = Organización + Hardware

Organización

Se refiere a los detalles de implementación de la ISA. Es decir:

- Organización e interconexión de la memoria.
- Diseño de los diferentes bloques de la CPU y para implementar el set de instrucciones.
- Implementación de paralelismo a nivel de Instrucciones, y/o de datos.
- Podemos así encontrar procesadores que poseen la misma ISA pero una organización muy diferente.

Ejemplo:

Los procesadores AMD FX y los Intel Core i7, tienen la misma ISA, sin embargo organizan su caché y su motor de ejecución de maneras diferentes.

Microarquitectura = Organización + Hardware

Hardware

Microarquitectura = Organización + Hardware

Hardware

- Se refiere a los detalles de diseño lógico y tecnología de fabricación.

Microarquitectura = Organización + Hardware

Hardware

- Se refiere a los detalles de diseño lógico y tecnología de fabricación.
- Existirán por lo tanto, procesadores con la misma ISA y la misma organización, pero absolutamente distintos a nivel de hardware y diseño lógico detallado.

Microarquitectura = Organización + Hardware

Hardware

- Se refiere a los detalles de diseño lógico y tecnología de fabricación.
- Existirán por lo tanto, procesadores con la misma ISA y la misma organización, pero absolutamente distintos a nivel de hardware y diseño lógico detallado.

Ejemplo:

El Pentium 4, diseñado para desktop, y el Pentium 4 M para notebooks. Este último tiene una cantidad de lógica para control del consumo de energía, siendo su hardware muy diferente del del Pentium 4 desktop.

¿Porque necesitamos saber todo esto?

¿Porque necesitamos saber todo esto?

- Para comprender lo que ocurre debajo del software.

¿Porque necesitamos saber todo esto?

- Para comprender lo que ocurre debajo del software.
- Comprender como los diseños de hardware impactan en el software y en el programador.

¿Porque necesitamos saber todo esto?

- Para comprender lo que ocurre debajo del software.
- Comprender como los diseños de hardware impactan en el software y en el programador.
- Estar en condiciones de cruzar verticalmente las capas que separan el silicio de la aplicación.

¿Porque necesitamos saber todo esto?

- Para comprender lo que ocurre debajo del software.
- Comprender como los diseños de hardware impactan en el software y en el programador.
- Estar en condiciones de cruzar verticalmente las capas que separan el silicio de la aplicación.
- Comprender las nociones básicas que rigen el diseño de un sistema de cómputo moderno.

1 Pioneros

2 Evolución

3 Arquitectura y Organización

4 Implementando la arquitectura

- El Procesador
- Implementación del Procesador
- Pipeline
- La memoria. El problema del acceso

Datapath, Unidades de Cálculo, y Control

Datapath, Unidades de Cálculo, y Control

- Sea cual sea el set de instrucciones que se vaya a implementar, hay dos actividades que inicialmente cualquier instrucción debe realizar:

Datapath, Unidades de Cálculo, y Control

- Sea cual sea el set de instrucciones que se vaya a implementar, hay dos actividades que inicialmente cualquier instrucción debe realizar:
  1. Apuntar el Program Counter (**PC**) a la posición de memoria que contiene la siguiente instrucción.

Datapath, Unidades de Cálculo, y Control

- Sea cual sea el set de instrucciones que se vaya a implementar, hay dos actividades que inicialmente cualquier instrucción debe realizar:
  1. Apuntar el Program Counter (**PC**) a la posición de memoria que contiene la siguiente instrucción.
  2. Leer uno o dos registros (dependiendo de la instrucción) contenidos en la palabra de instrucción. Ej: `ld` o `lw` requieren leer solo un registro: el que contiene la dirección de memoria que hay que leer. Otros tipos de instrucción leen dos registros.

Datapath, Unidades de Cálculo, y Control

- Sea cual sea el set de instrucciones que se vaya a implementar, hay dos actividades que inicialmente cualquier instrucción debe realizar:
  1. Apuntar el Program Counter (**PC**) a la posición de memoria que contiene la siguiente instrucción.
  2. Leer uno o dos registros (dependiendo de la instrucción) contenidos en la palabra de instrucción. Ej: `ld` o `lw` requieren leer solo un registro: el que contiene la dirección de memoria que hay que leer. Otros tipos de instrucción leen dos registros.
- Las acciones posteriores del procesador dependen de cada instrucción específica.

Datapath, Unidades de Cálculo, y Control

- Sea cual sea el set de instrucciones que se vaya a implementar, hay dos actividades que inicialmente cualquier instrucción debe realizar:
  1. Apuntar el Program Counter (**PC**) a la posición de memoria que contiene la siguiente instrucción.
  2. Leer uno o dos registros (dependiendo de la instrucción) contenidos en la palabra de instrucción. Ej: `ld` o `lw` requieren leer solo un registro: el que contiene la dirección de memoria que hay que leer. Otros tipos de instrucción leen dos registros.
- Las acciones posteriores del procesador dependen de cada instrucción específica.
- No obstante cuanto mas simple sea el set de instrucciones (RISC), el paso siguiente será el mismo para un porcentaje mayor de instrucciones.

Datapath, Unidades de Cálculo, y Control

- Sea cual sea el set de instrucciones que se vaya a implementar, hay dos actividades que inicialmente cualquier instrucción debe realizar:
  1. Apuntar el Program Counter (**PC**) a la posición de memoria que contiene la siguiente instrucción.
  2. Leer uno o dos registros (dependiendo de la instrucción) contenidos en la palabra de instrucción. Ej: `ld` o `lw` requieren leer solo un registro: el que contiene la dirección de memoria que hay que leer. Otros tipos de instrucción leen dos registros.
- Las acciones posteriores del procesador dependen de cada instrucción específica.
- No obstante cuanto mas simple sea el set de instrucciones (RISC), el paso siguiente será el mismo para un porcentaje mayor de instrucciones.
- Luego de los dos pasos anteriores un porcentaje de instrucciones utiliza la ALU.

Datapath, Unidades de Cálculo, y Control

- Sea cual sea el set de instrucciones que se vaya a implementar, hay dos actividades que inicialmente cualquier instrucción debe realizar:
  1. Apuntar el Program Counter (**PC**) a la posición de memoria que contiene la siguiente instrucción.
  2. Leer uno o dos registros (dependiendo de la instrucción) contenidos en la palabra de instrucción. Ej: `ld` o `lw` requieren leer solo un registro: el que contiene la dirección de memoria que hay que leer. Otros tipos de instrucción leen dos registros.
- Las acciones posteriores del procesador dependen de cada instrucción específica.
- No obstante cuanto mas simple sea el set de instrucciones (RISC), el paso siguiente será el mismo para un porcentaje mayor de instrucciones.
- Luego de los dos pasos anteriores un porcentaje de instrucciones utiliza la ALU.
- Las instrucciones de referencia a memoria (load, store y branch incondicional), utilizan la ALU para calcular la dirección de memoria a acceder, las instrucciones artitméticas y lógicas utilizarán la ALU para ejecutar la instrucción solicitada, los saltos condicionales para evaluar la condición del salto.

1 Pioneros

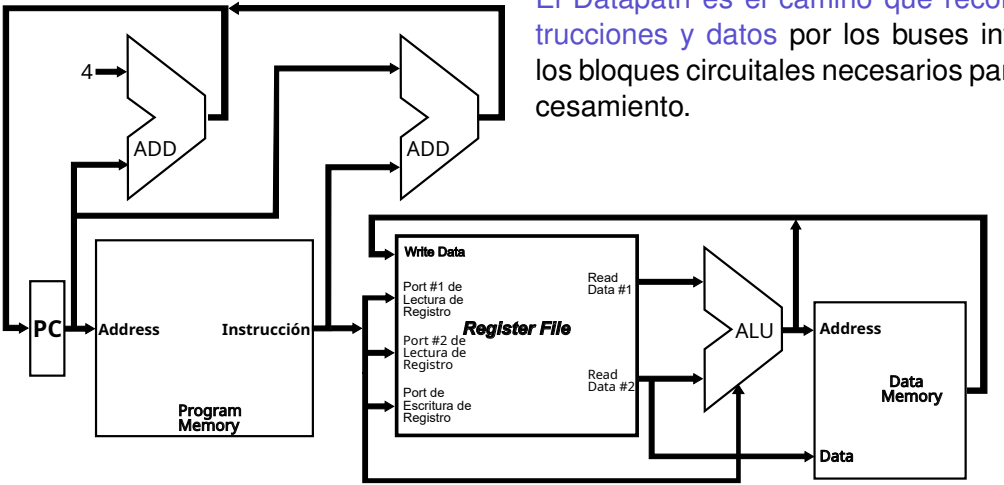
2 Evolución

3 Arquitectura y Organización

4 Implementando la arquitectura

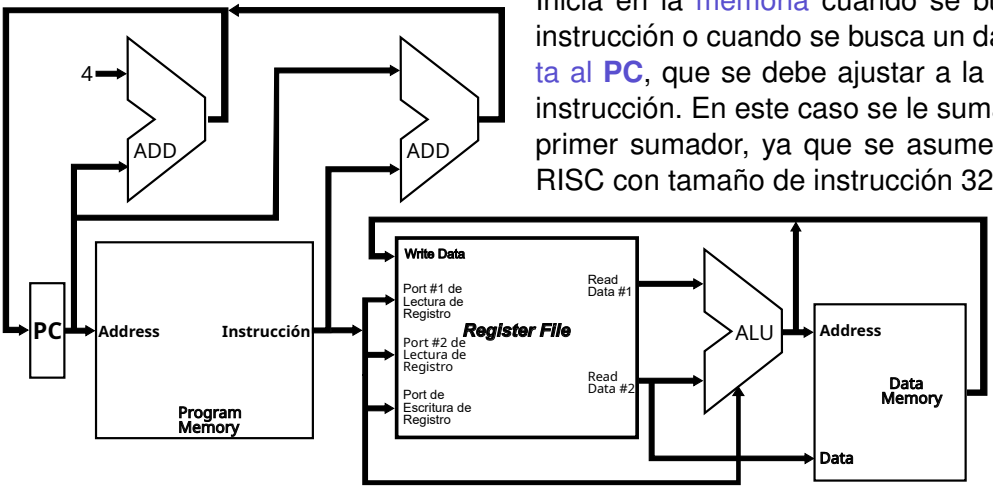
- El Procesador
- Implementación del Procesador
- Pipeline
- La memoria. El problema del acceso

Diseño de un DataPath Simple



El Datapath es el camino que recorren instrucciones y datos por los buses internos y los bloques circuitales necesarios para su procesamiento.

Diseño de un DataPath Simple



Inicia en la memoria cuando se busca una instrucción o cuando se busca un dato. Afecta al **PC**, que se debe ajustar a la siguiente instrucción. En este caso se le suma 4, en el primer sumador, ya que se asume un CPU RISC con tamaño de instrucción 32 bit

El Register File

El Register File

- Una estructura fundamental en el **Datapath** es el Register File.

El Register File

- Una estructura fundamental en el **Datapath** es el Register File.
- Un Register File de tamaño **n**, consiste en un arreglo de **n** registros, numerados de **0** a **n-1**, que se pueden leer y escribir proporcionando tan solo el número de registro al que se accede.

El Register File

- Una estructura fundamental en el **Datapath** es el Register File.
- Un Register File de tamaño **n**, consiste en un arreglo de **n** registros, numerados de **0** a **n-1**, que se pueden leer y escribir proporcionando tan solo el número de registro al que se accede.
- Se puede implementar con un decodificador para cada puerto de lectura o escritura y un arreglo de registros construida a partir de Flip-Flop's D.

El Register File

- Una estructura fundamental en el **Datapath** es el Register File.
- Un Register File de tamaño **n**, consiste en un arreglo de **n** registros, numerados de **0** a **n-1**, que se pueden leer y escribir proporcionando tan solo el número de registro al que se accede.
- Se puede implementar con un decodificador para cada puerto de lectura o escritura y un arreglo de registros construida a partir de Flip-Flop's D.
- Dado que la lectura de un registro no cambia ningún estado, y la salida Q de un FF está disponible, solo necesitamos proporcionar un número de registro como entrada, y la única salida serán los datos contenidos en ese registro.

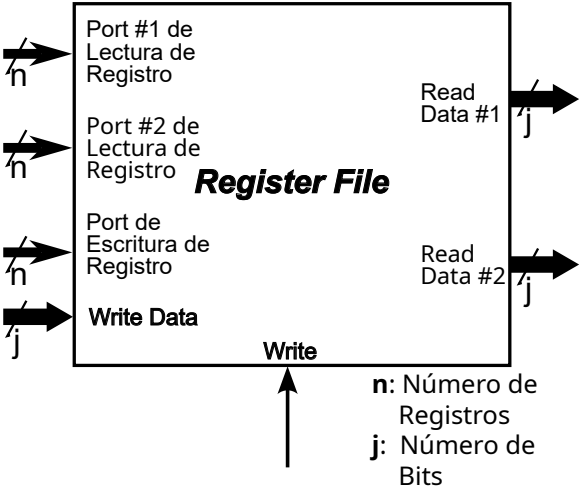
El Register File

- Una estructura fundamental en el **Datapath** es el Register File.
- Un Register File de tamaño **n**, consiste en un arreglo de **n** registros, numerados de **0** a **n-1**, que se pueden leer y escribir proporcionando tan solo el número de registro al que se accede.
- Se puede implementar con un decodificador para cada puerto de lectura o escritura y un arreglo de registros construida a partir de Flip-Flop's D.
- Dado que la lectura de un registro no cambia ningún estado, y la salida Q de un FF está disponible, solo necesitamos proporcionar un número de registro como entrada, y la única salida serán los datos contenidos en ese registro.
- Para escribir un registro, necesitaremos tres entradas: el número de registro dentro del arreglo, los datos a escribir y un **clock** que controla la escritura en el registro.

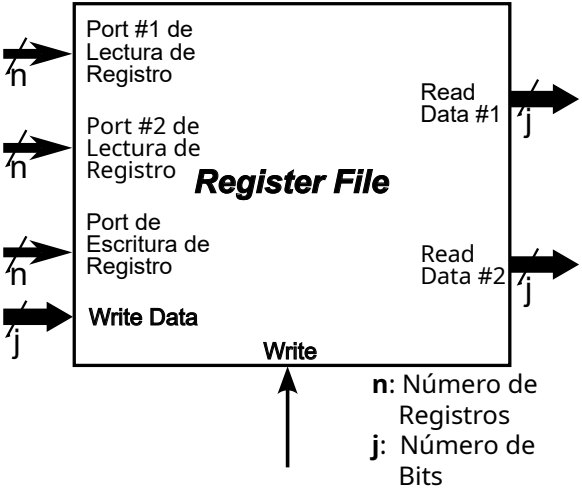
El Register File

- Una estructura fundamental en el **Datapath** es el Register File.
- Un Register File de tamaño **n**, consiste en un arreglo de **n** registros, numerados de **0** a **n-1**, que se pueden leer y escribir proporcionando tan solo el número de registro al que se accede.
- Se puede implementar con un decodificador para cada puerto de lectura o escritura y un arreglo de registros construida a partir de Flip-Flop's D.
- Dado que la lectura de un registro no cambia ningún estado, y la salida Q de un FF está disponible, solo necesitamos proporcionar un número de registro como entrada, y la única salida serán los datos contenidos en ese registro.
- Para escribir un registro, necesitaremos tres entradas: el número de registro dentro del arreglo, los datos a escribir y un **clock** que controla la escritura en el registro.
- Utilizamos un Register File con dos puertos de lectura y uno de escritura.
  Se muestra en el slide siguiente.

El Register File

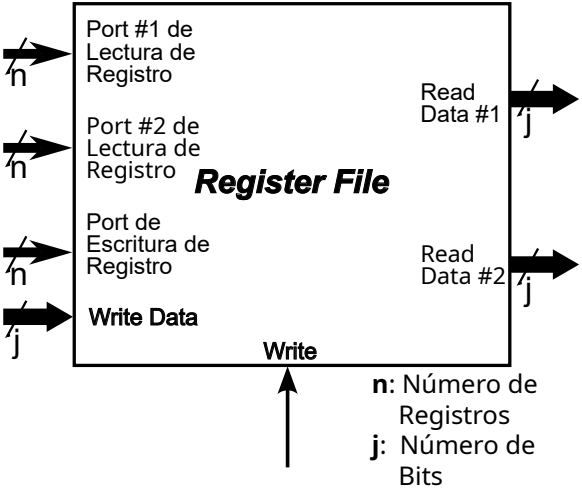


El Register File



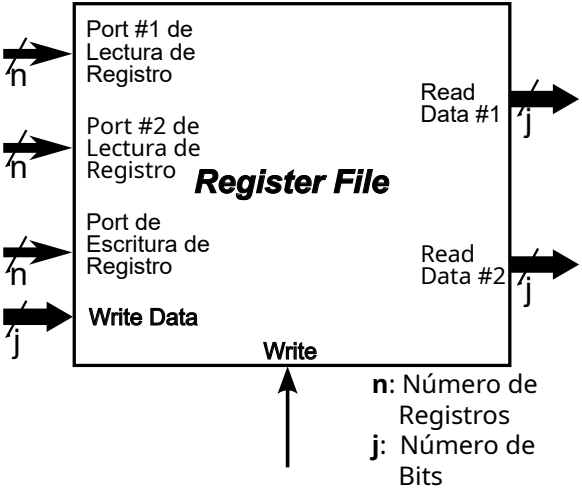
- Se representa el símbolo de un Register File de **n**x**j** (**n** registros de **j**bit c/u).

El Register File



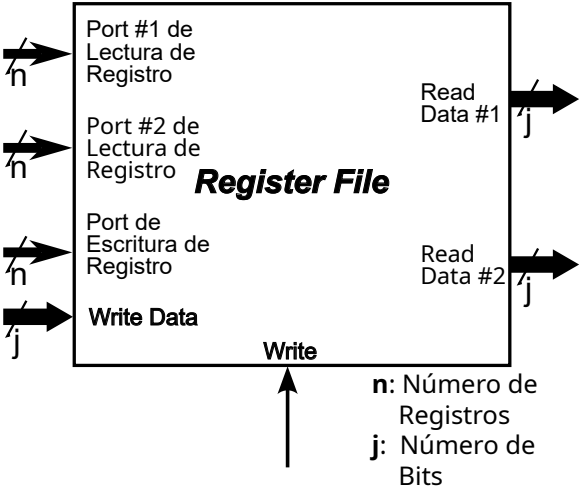
- Se representa el símbolo de un Register File de **n**x**j** (**n** registros de **j**bit c/u).
- Para evitar atascos y agilizar el procesamiento es conveniente al menos dos puertos de lectura.

El Register File



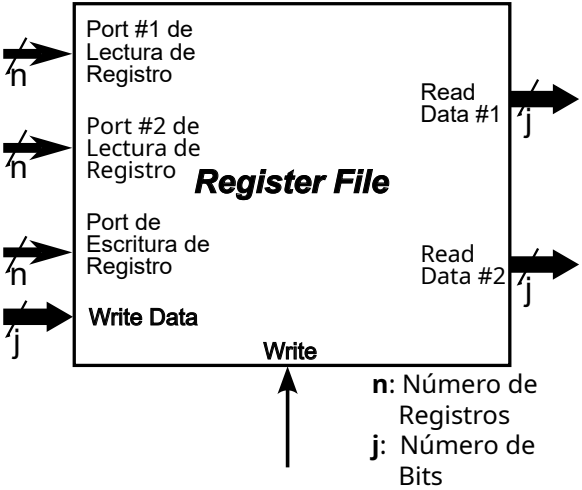
- Se representa el símbolo de un Register File de **n**x**j** (**n** registros de **j**bit c/u).
- Para evitar atascos y agilizar el procesamiento es conveniente al menos dos puertos de lectura.
- Puede leerse los dos operandos de una instrucción de una sola vez.

El Register File



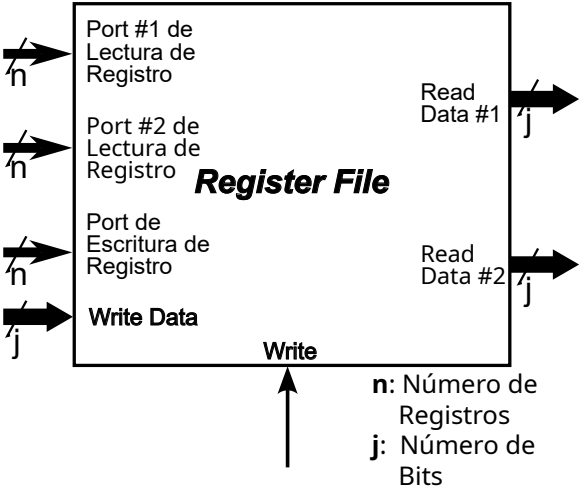
- Se representa el símbolo de un Register File de **n**x**j** (**n** registros de **j**bit c/u).
- Para evitar atascos y agilizar el procesamiento es conveniente al menos dos puertos de lectura.
- Puede leerse los dos operandos de una instrucción de una sola vez.
- Un puerto es una entrada de **n** bits al que se envía el número de registro cuyo valor se desea leer.

El Register File



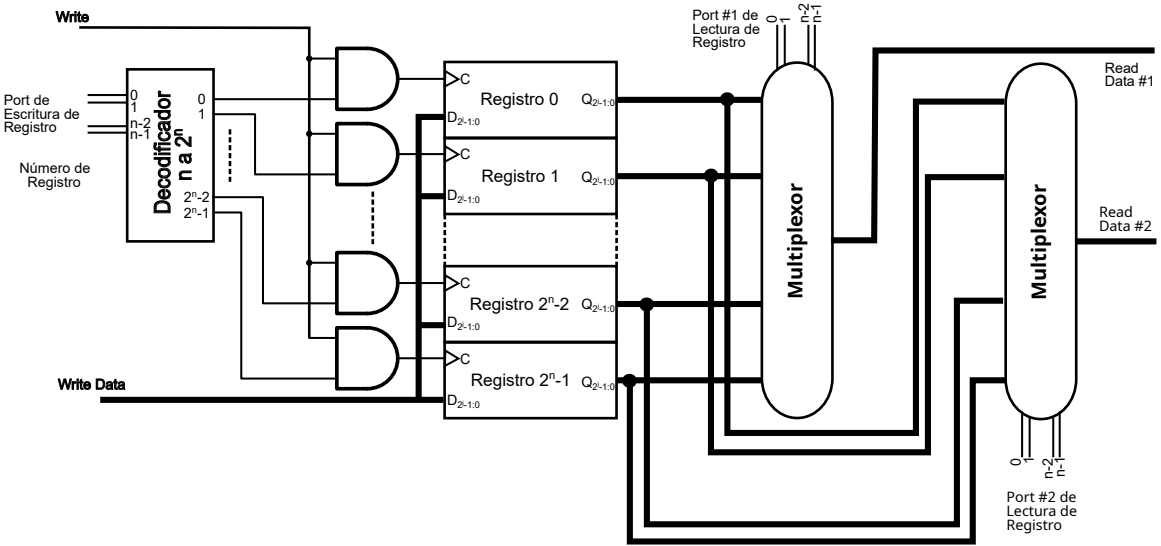
- Se representa el símbolo de un Register File de **n**x**j** (**n** registros de **j**bit c/u).
- Para evitar atascos y agilizar el procesamiento es conveniente al menos dos puertos de lectura.
- Puede leerse los dos operandos de una instrucción de una sola vez.
- Un puerto es una entrada de **n** bits al que se envía el número de registro cuyo valor se desea leer.
- El contenido del registro sale por las **j** líneas **`Read Data #1`** o **`Read Data #2`** según el puerto seleccionado.

El Register File



- Se representa el símbolo de un Register File de **n**x**j** (**n** registros de **j**bit c/u).
- Para evitar atascos y agilizar el procesamiento es conveniente al menos dos puertos de lectura.
- Puede leerse los dos operandos de una instrucción de una sola vez.
- Un puerto es una entrada de **n** bits al que se envía el número de registro cuyo valor se desea leer.
- El contenido del registro sale por las **j** líneas **`Read Data #1`** o **`Read Data #2`** según el puerto seleccionado.
- Un solo puerto para escritura.

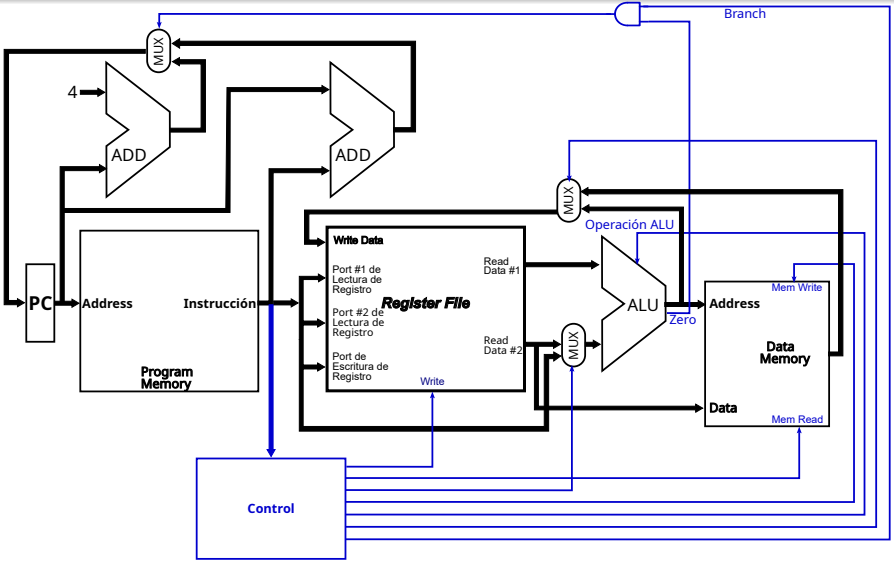
El Register File en detalle



El Register File en Verilog (behavioral)

```
    reg [31:0] A;
    wire [31:0] b;

    always @(posedge clock) A<=b:

module registerfile (Read1, Read2, WriteReg, WriteData, RegWrite, Data1, Data2, clock);
    input[5:0] Read1, Read2, WriteReg; // Cant Registros a leer o escribir
    input[31:0] WriteData; // Puerto de escritura
    input RegWrite; //Control de escritura
    clock ; //Línea de clock para disparar la escritura
    output[31:0] Data1, Data2; //Puertos de lectura de registros
    reg[31:0] RF[31:0] ; //32 Registros de 32 bits

    assign Data1 = RF[Read1];
    assign Data2 = RF[Read2];

    always begin
      //Escribir el nuevo valor en el registro si REgWrite es alto
     @(posedge clock) if (RegWrite) RF[WriteReg] <= WriteData
    end
endmodule
```

Diseño de un DataPath Simple



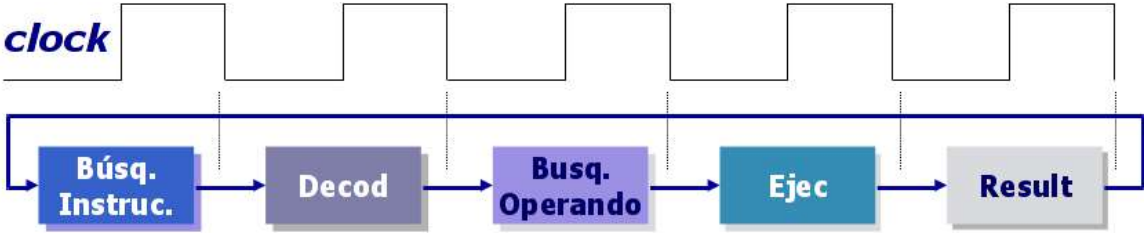
1 Pioneros

2 Evolución

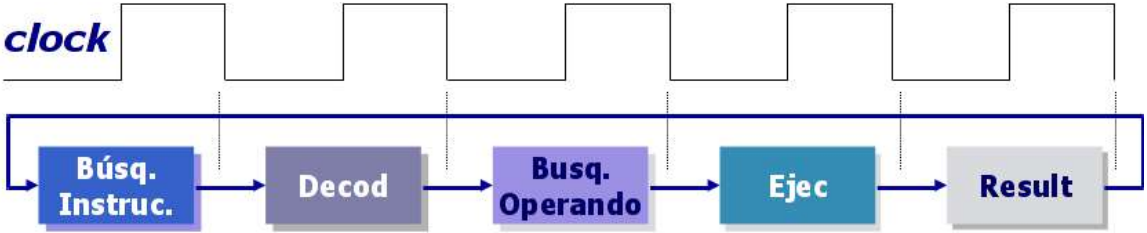
3 Arquitectura y Organización

4 Implementando la arquitectura
- El Procesador
- Implementación del Procesador
- Pipeline
- La memoria. El problema del acceso

Máquina de estados elemental

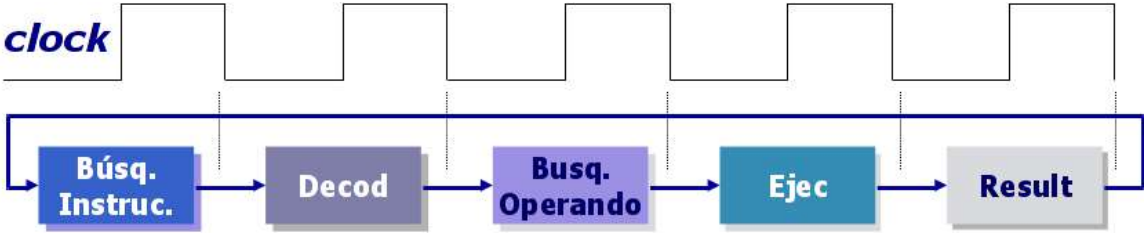


Máquina de estados elemental



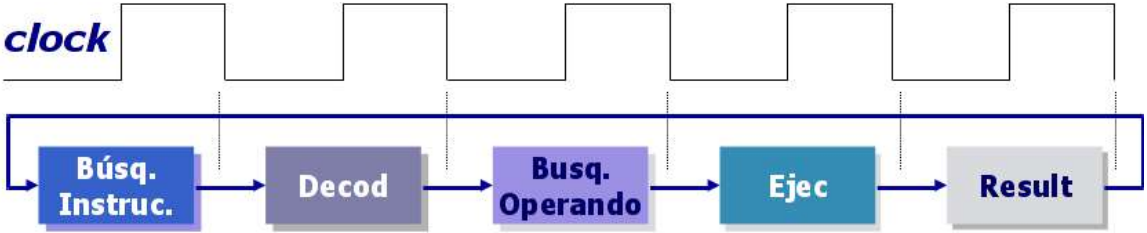
- Un procesador ejecuta una máquina de estados compuesta por diferentes etapas.

Máquina de estados elemental



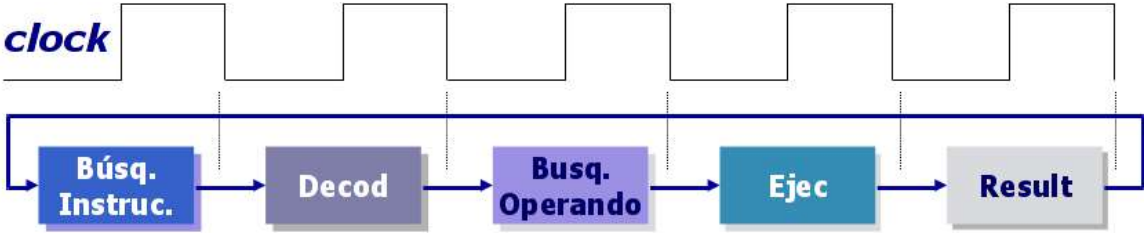
- Un procesador ejecuta una máquina de estados compuesta por diferentes etapas.
- En las primeras generaciones de microprocesadores cada etapa se ejecutaba durante uno o mas ciclos de clock, durante los cuales la CPU entera estaba dedicada a ella.

Máquina de estados elemental



- Un procesador ejecuta una máquina de estados compuesta por diferentes etapas.
- En las primeras generaciones de microprocesadores cada etapa se ejecutaba durante uno o mas ciclos de clock, durante los cuales la CPU entera estaba dedicada a ella.
- Una vez finalizada una etapa, en el siguiente ciclo de clock se pasaba a ejecutar la siguiente.

Máquina de estados elemental



- Un procesador ejecuta una máquina de estados compuesta por diferentes etapas.
- En las primeras generaciones de microprocesadores cada etapa se ejecutaba durante uno o mas ciclos de clock, durante los cuales la CPU entera estaba dedicada a ella.
- Una vez finalizada una etapa, en el siguiente ciclo de clock se pasaba a ejecutar la siguiente.
- De este modo, ejecutar una instrucción insumía varios ciclos de clock.

Pipeline

- Permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.

Pipeline

- Permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.
- Formaliza el concepto de ***Instruction Level Paralelism (ILP)***.

Pipeline

- Permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.
- Formaliza el concepto de ***Instruction Level Paralelism (ILP)***.
- Requiere muy poco o ningún hardware adicional.

Pipeline

- Permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.
- Formaliza el concepto de ***Instruction Level Paralelism (ILP)***.
- Requiere muy poco o ningún hardware adicional.
- Solo necesita que los bloques del procesador que resuelven la máquina de estados de ejecución de una instrucción, operen en forma simultánea.

Pipeline

- Permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.
- Formaliza el concepto de ***Instruction Level Paralelism (ILP)***.
- Requiere muy poco o ningún hardware adicional.
- Solo necesita que los bloques del procesador que resuelven la máquina de estados de ejecución de una instrucción, operen en forma simultánea.
- Se logra si todos los bloques funcionales trabajan en paralelo, pero cada uno en una instrucción diferente.

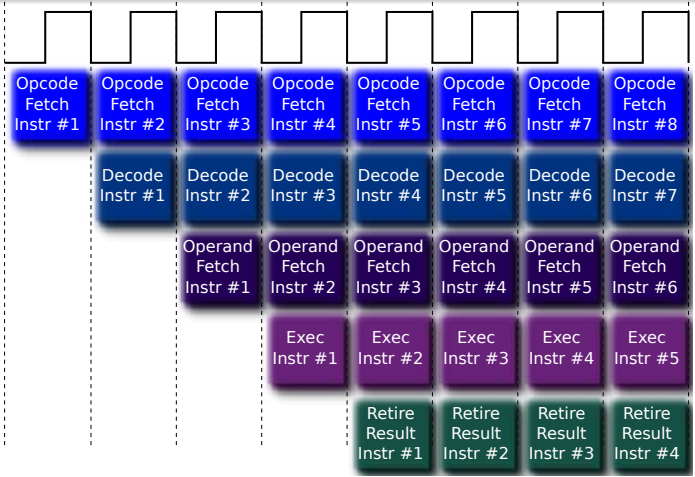
Pipeline

- Permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.
- Formaliza el concepto de ***Instruction Level Paralelism (ILP)***.
- Requiere muy poco o ningún hardware adicional.
- Solo necesita que los bloques del procesador que resuelven la máquina de estados de ejecución de una instrucción, operen en forma simultánea.
- Se logra si todos los bloques funcionales trabajan en paralelo, pero cada uno en una instrucción diferente.
- Es algo parecido al concepto de una línea de montaje, en donde cada operación se descompone en partes, y se ejecutan en un mismo momento diferentes partes de diferentes operaciones.

Pipeline

- Permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.
- Formaliza el concepto de ***Instruction Level Paralelism (ILP)***.
- Requiere muy poco o ningún hardware adicional.
- Solo necesita que los bloques del procesador que resuelven la máquina de estados de ejecución de una instrucción, operen en forma simultánea.
- Se logra si todos los bloques funcionales trabajan en paralelo, pero cada uno en una instrucción diferente.
- Es algo parecido al concepto de una línea de montaje, en donde cada operación se descompone en partes, y se ejecutan en un mismo momento diferentes partes de diferentes operaciones.
- Cada parte se denomina etapa (**stage**)

Pipeline



Este modelo es teórico y representa la situación ideal.

Pipeline

Conclusiones Preliminares

Pipeline

Conclusiones Preliminares

1. El pipeline llega a su condición de régimen luego de tantos ciclos de clock como etapas tenga.

Pipeline

Conclusiones Preliminares

1. El pipeline llega a su condición de régimen luego de tantos ciclos de clock como etapas tenga.
2. En un pipeline de 5 etapas, y en el caso ideal en que cada etapa consuma solamente un ciclo de clock, una arquitectura pipeline provee un resultado de instrucción por cada ciclo de clock, a partir del ciclo de clock en el cual se llega a resolver la primer instrucción.

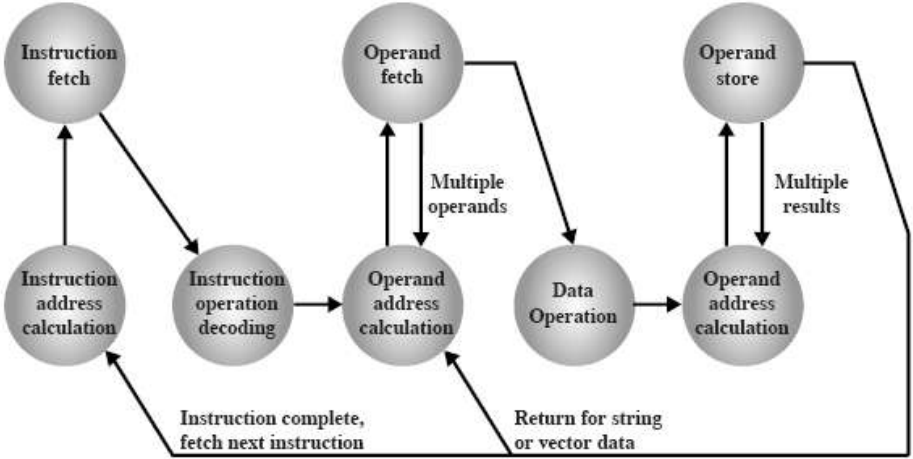
Pipeline

Conclusiones Preliminares

1. El pipeline llega a su condición de régimen luego de tantos ciclos de clock como etapas tenga.
2. En un pipeline de 5 etapas, y en el caso ideal en que cada etapa consuma solamente un ciclo de clock, una arquitectura pipeline provee un resultado de instrucción por cada ciclo de clock, a partir del ciclo de clock en el cual se llega a resolver la primer instrucción.
3. Este escenario permanente es puramente teórico. En la práctica no se cumple todo el tiempo.

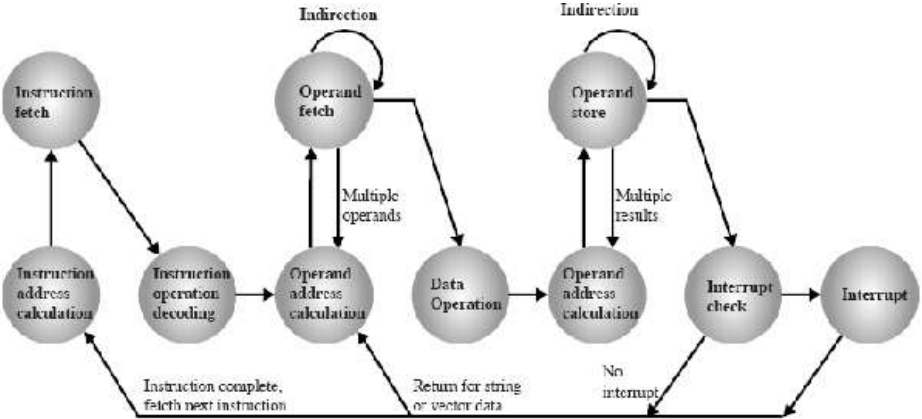
Profundidad del Pipeline

Si las conclusiones son ciertas agregar etapas mejoraría la performance



Mas y mas etapas...

Etapas mas simples aseguran que trabajan en un solo clock. Por eso podríamos pensar en aumentarlas mas y mas . . .



Eficiencia de un pipeline

Eficiencia de un pipeline

- ¿Porque querríamos aumentar y aumentar el número de etapas?

Eficiencia de un pipeline

- **¿Porque querríamos aumentar y aumentar el número de etapas?**

- Para un tiempo conocido de procesamiento interno de una instrucción para una arquitectura “no pipeline”, intuitivamente puede comprenderse que en principio cuantas mas etapas podamos definir para ejecutar esta operación, al ponerlas a trabajar a todas en paralelo en un pipeline, el tiempo de ejecución de la instrucción se reducirá proporcionalmente con la cantidad de etapas.

Eficiencia de un pipeline

- **¿Porque querríamos aumentar y aumentar el número de etapas?**

- Para un tiempo conocido de procesamiento interno de una instrucción para una arquitectura “no pipeline”, intuitivamente puede comprenderse que en principio cuantas mas etapas podamos definir para ejecutar esta operación, al ponerlas a trabajar a todas en paralelo en un pipeline, el tiempo de ejecución de la instrucción se reducirá proporcionalmente con la cantidad de etapas.

- Si llamamos ***TPI*** (Time Per Instruction) al tiempo medido entre el resultado de dos instrucciones consecutivas, la siguiente expresión resume todo el análisis.

$$TPI = \frac{Tiempo\ por\ instrucción\ en\ la\ CPU\ ''\ No-Pipeline''}{Cantidad\ de\ etapas}$$

Conclusiones

Conclusiones

- La situación teórica e ideal dada por la ecuación anterior no es 100 % cierta.

Conclusiones

- La situación teórica e ideal dada por la ecuación anterior no es 100 % cierta.
- En la práctica existen overheads introducidos por el hardware de implementación del pipeline, que suman pequeñas demoras en la interfaz entre cada una de sus etapas. La realidad es que si bien existen, estas demoras resultan suficientemente pequeñas como para aceptar que el ***TPI*** así calculado, se aproxima mucho al ideal.

Conclusiones

- La situación teórica e ideal dada por la ecuación anterior no es 100 % cierta.
- En la práctica existen overheads introducidos por el hardware de implementación del pipeline, que suman pequeñas demoras en la interfaz entre cada una de sus etapas. La realidad es que si bien existen, estas demoras resultan suficientemente pequeñas como para aceptar que el ***TPI*** así calculado, se aproxima mucho al ideal.
- El resultado es que el efecto del pipeline en la práctica se percibe en una cantidad de ciclos de clock significativamente menor para ejecutar cada instrucción.

Conclusiones

- La situación teórica e ideal dada por la ecuación anterior no es 100 % cierta.
- En la práctica existen overheads introducidos por el hardware de implementación del pipeline, que suman pequeñas demoras en la interfaz entre cada una de sus etapas. La realidad es que si bien existen, estas demoras resultan suficientemente pequeñas como para aceptar que el ***TPI*** así calculado, se aproxima mucho al ideal.
- El resultado es que el efecto del pipeline en la práctica se percibe en una cantidad de ciclos de clock significativamente menor para ejecutar cada instrucción.
- El pipeline no reduce el tiempo de ejecución de cada instrucción individual, sino que al aplicarse en paralelo al flujo de instrucciones, incrementa el número de instrucciones completadas por unidad de tiempo.

Conclusiones

- La situación teórica e ideal dada por la ecuación anterior no es 100 % cierta.
- En la práctica existen overheads introducidos por el hardware de implementación del pipeline, que suman pequeñas demoras en la interfaz entre cada una de sus etapas. La realidad es que si bien existen, estas demoras resultan suficientemente pequeñas como para aceptar que el ***TPI*** así calculado, se aproxima mucho al ideal.
- El resultado es que el efecto del pipeline en la práctica se percibe en una cantidad de ciclos de clock significativamente menor para ejecutar cada instrucción.
- El pipeline no reduce el tiempo de ejecución de cada instrucción individual, sino que al aplicarse en paralelo al flujo de instrucciones, incrementa el número de instrucciones completadas por unidad de tiempo.
- De hecho el overhead del pipeline perjudica el tiempo de ejecución individual, aunque de manera poco significativa, al agregar un $\delta.t$ a cada instrucción.

Conclusiones

- La situación teórica e ideal dada por la ecuación anterior no es 100 % cierta.
- En la práctica existen overheads introducidos por el hardware de implementación del pipeline, que suman pequeñas demoras en la interfaz entre cada una de sus etapas. La realidad es que si bien existen, estas demoras resultan suficientemente pequeñas como para aceptar que el ***TPI*** así calculado, se aproxima mucho al ideal.
- El resultado es que el efecto del pipeline en la práctica se percibe en una cantidad de ciclos de clock significativamente menor para ejecutar cada instrucción.
- El pipeline no reduce el tiempo de ejecución de cada instrucción individual, sino que al aplicarse en paralelo al flujo de instrucciones, incrementa el número de instrucciones completadas por unidad de tiempo.
- De hecho el overhead del pipeline perjudica el tiempo de ejecución individual, aunque de manera poco significativa, al agregar un $\delta.t$ a cada instrucción.
- El rendimiento (throughput) del procesador mejora notablemente ya que los programas ejecutan notablemente mas rápido.

Obstáculos de un pipeline

Obstáculos de un pipeline

- Hay obstáculos que conspiran contra la eficiencia de un pipeline.

Obstáculos de un pipeline

- Hay obstáculos que conspiran contra la eficiencia de un pipeline.
- Podemos agruparlos en tres categorías:

Obstáculos de un pipeline

- Hay obstáculos que conspiran contra la eficiencia de un pipeline.
- Podemos agruparlos en tres categorías:

1. ***Obstáculos estructurales***.

Obstáculos de un pipeline

- Hay obstáculos que conspiran contra la eficiencia de un pipeline.
- Podemos agruparlos en tres categorías:

1. ***Obstáculos estructurales***.
2. ***Obstáculos de datos***.

Obstáculos de un pipeline

- Hay obstáculos que conspiran contra la eficiencia de un pipeline.
- Podemos agruparlos en tres categorías:

1. ***Obstáculos estructurales***.
2. ***Obstáculos de datos***.
3. ***Obstáculos de control***.

Obstáculos de un pipeline

- Hay obstáculos que conspiran contra la eficiencia de un pipeline.
- Podemos agruparlos en tres categorías:

  1. ***Obstáculos estructurales***.
  2. ***Obstáculos de datos***.
  3. ***Obstáculos de control***.

- Al efecto ocasionado por un obstácuo se lo denomina ***pipeline stall***.

Obstáculos de un pipeline

- Hay obstáculos que conspiran contra la eficiencia de un pipeline.
- Podemos agruparlos en tres categorías:

  1. ***Obstáculos estructurales***.
  2. ***Obstáculos de datos***.
  3. ***Obstáculos de control***.

- Al efecto ocasionado por un obstácuo se lo denomina ***pipeline stall***.
- Su efecto degrada la performance del procesador.

Obstáculos Estructurales

Se pueden dar por varias razones:

- Una etapa no está suficientemente atomizada, y concentra aún demasiadas funciones lo que hace que para completarse requiere mas de un ciclo de clock.

Obstáculos Estructurales

Se pueden dar por varias razones:

- Una etapa no está suficientemente atomizada, y concentra aún demasiadas funciones lo que hace que para completarse requiere mas de un ciclo de clock.
- Si dos instrucciones que utilizarán esta etapa están mas próximas del tiempo que necesita esta etapa para procesar su parte, caerán en conflicto de recursos para su ejecución.

Obstáculos Estructurales

Se pueden dar por varias razones:

- Una etapa no está suficientemente atomizada, y concentra aún demasiadas funciones lo que hace que para completarse requiere mas de un ciclo de clock.
- Si dos instrucciones que utilizarán esta etapa están mas próximas del tiempo que necesita esta etapa para procesar su parte, caerán en conflicto de recursos para su ejecución.
- Este grupo de instrucciones entonces, no puede pasar por esa etapa del pipeline en un ciclo de clock.

Obstáculos Estructurales

Se pueden dar por varias razones:

- Una etapa no está suficientemente atomizada, y concentra aún demasiadas funciones lo que hace que para completarse requiere mas de un ciclo de clock.
- Si dos instrucciones que utilizarán esta etapa están mas próximas del tiempo que necesita esta etapa para procesar su parte, caerán en conflicto de recursos para su ejecución.
- Este grupo de instrucciones entonces, no puede pasar por esa etapa del pipeline en un ciclo de clock.
- En estas circunstancias una de las instrucciones debe detenerse. Como consecuencia CPI se incrementa en 1 o mas respecto del valor ideal.

Obstáculos Estructurales: Ejemplo

- Tenemos un procesador que solo tiene una etapa para acceder a memoria y la comparte para acceso a datos e instrucciones.

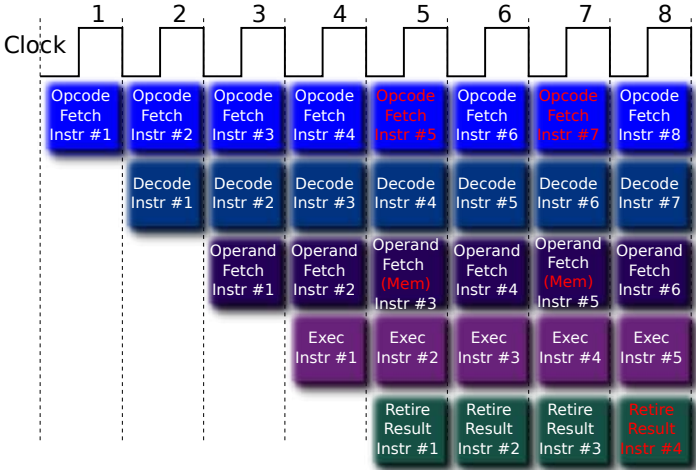
Obstáculos Estructurales: Ejemplo

- Tenemos un procesador que solo tiene una etapa para acceder a memoria y la comparte para acceso a datos e instrucciones.
- En el caso de que se necesite un operando de memoria, el acceso para traer este operando interferirá con la búsqueda del operando de una instrucción mas adelante del programa.

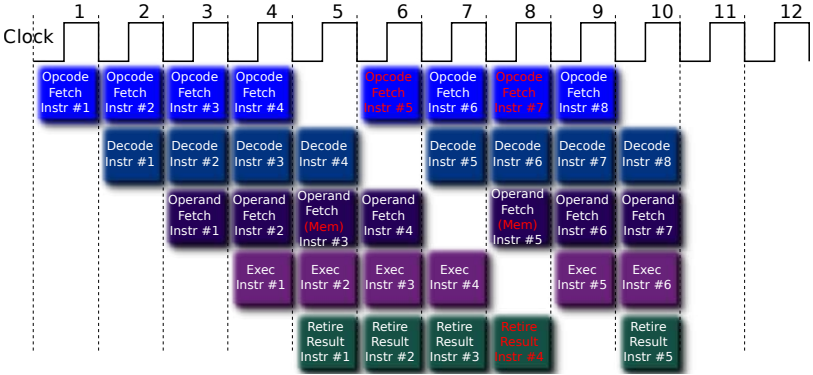
Obstáculos Estructurales: Ejemplo

- Tenemos un procesador que solo tiene una etapa para acceder a memoria y la comparte para acceso a datos e instrucciones.
- En el caso de que se necesite un operando de memoria, el acceso para traer este operando interferirá con la búsqueda del operando de una instrucción mas adelante del programa.
- También interferirá con el Fetch de la siguiente instrucción

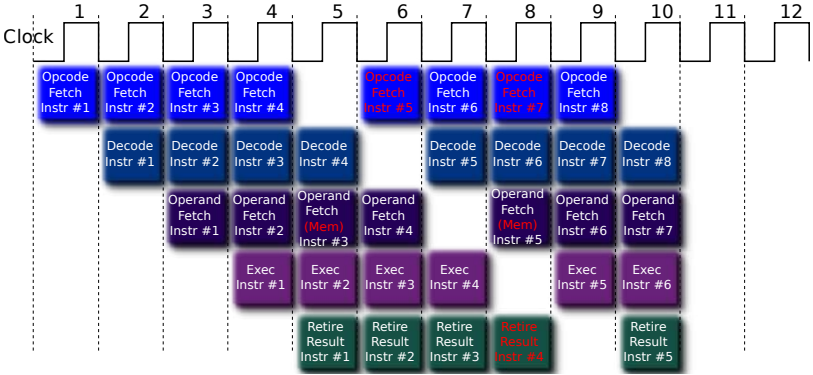
Obstáculos Estructurales: Accesos concurrentes a memoria



Obstáculos Estructurales - Efecto en CPI

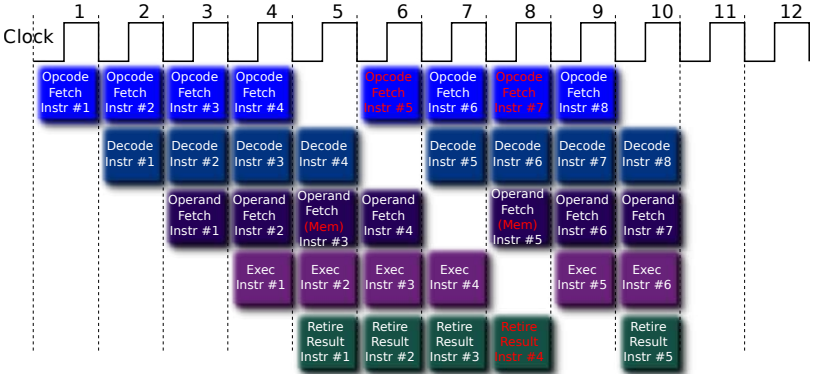


Obstáculos Estructurales - Efecto en CPI



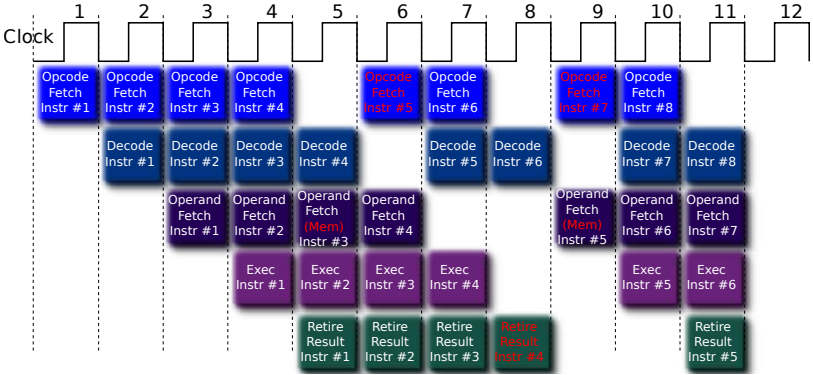
- Por cada Obstáculo, pospone una operación. $CPI = CPI + 1$

Obstáculos Estructurales - Efecto en CPI



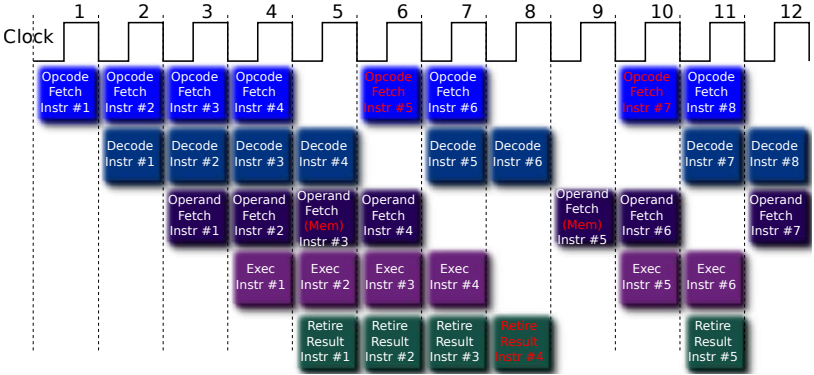
- Por cada Obstáculo, pospone una operación. $CPI = CPI + 1$
- En general la cantidad de CPI que se incrementan es igual a la cantidad de concurrencias menos 1, en el lapso considerado

Obstáculos Estructurales - Efecto en CPI



- Por cada Obstáculo, pospone una operación. $CPI = CPI + 1$
- En general la cantidad de CPI que se incrementan es igual a la cantidad de concurrencias menos 1, en el lapso considerado

Obstáculos Estructurales - Efecto en CPI



- Por cada Obstáculo, pospone una operación. $CPI = CPI + 1$
- En general la cantidad de CPI que se incrementan es igual a la cantidad de concurrencias menos 1, en el lapso considerado

Obstáculos Estructurales - Posibles soluciones

Obstáculos Estructurales - Posibles soluciones

- Cualesquiera sean las soluciones que deseemos implementar, es necesario agregar hardware.

Obstáculos Estructurales - Posibles soluciones

- Cualesquiera sean las soluciones que deseemos implementar, es necesario agregar hardware.
- En el caso de accesos a memoria, se puede resolver mediante las siguientes opciones por separado o combinadas:

Obstáculos Estructurales - Posibles soluciones

- Cualesquiera sean las soluciones que deseemos implementar, es necesario agregar hardware.
- En el caso de accesos a memoria, se puede resolver mediante las siguientes opciones por separado o combinadas:
  1. Desdoblamiento del cache L1 en Cache de datos y Cache de instrucciones.

Obstáculos Estructurales - Posibles soluciones

- Cualesquiera sean las soluciones que deseemos implementar, es necesario agregar hardware.
- En el caso de accesos a memoria, se puede resolver mediante las siguientes opciones por separado o combinadas:
  1. Desdoblamiento del cache L1 en Cache de datos y Cache de instrucciones.
  2. Empleo de buffers de instrucciones implementados como pequeñas colas FIFO.

Obstáculos Estructurales - Posibles soluciones

- Cualesquiera sean las soluciones que deseemos implementar, es necesario agregar hardware.
- En el caso de accesos a memoria, se puede resolver mediante las siguientes opciones por separado o combinadas:
  1. Desdoblamiento del cache L1 en Cache de datos y Cache de instrucciones.
  2. Empleo de buffers de instrucciones implementados como pequeñas colas FIFO.
  3. Ensanchamiento de los buses mas allá de los anchos de palabra del procesador.

Obstáculos Estructurales - Posibles soluciones

- Cualesquiera sean las soluciones que deseemos implementar, es necesario agregar hardware.
- En el caso de accesos a memoria, se puede resolver mediante las siguientes opciones por separado o combinadas:
  1. Desdoblamiento del cache L1 en Cache de datos y Cache de instrucciones.
  2. Empleo de buffers de instrucciones implementados como pequeñas colas FIFO.
  3. Ensanchamiento de los buses mas allá de los anchos de palabra del procesador.
- Aumentar la profundidad del pipeline, produce etapas mucho mas simples capaces de resolver en un clock su tarea.

Obstáculos de datos

Obstáculos de datos

- Se producen cuando por efecto del pipeline, una instrucción requiere de un dato antes de que este esté disponible por efecto de la secuencia lógica prevista en el programa.

Obstáculos de datos

- Se producen cuando por efecto del pipeline, una instrucción requiere de un dato antes de que este esté disponible por efecto de la secuencia lógica prevista en el programa.
- Consideremos el código que figura al pié. Corresponde a un procesador ARM.

Obstáculos de datos

- Se producen cuando por efecto del pipeline, una instrucción requiere de un dato antes de que este esté disponible por efecto de la secuencia lógica prevista en el programa.
- Consideremos el código que figura al pié. Corresponde a un procesador ARM.
- Se observa dependencias para el Registro R1. Hasta que la instrucción ***add*** no complete su operación, R1 no tiene un valor válido. Por lo tanto no puede continuar aplicándolo en las restantes que lo utilizan.

Obstáculos de datos

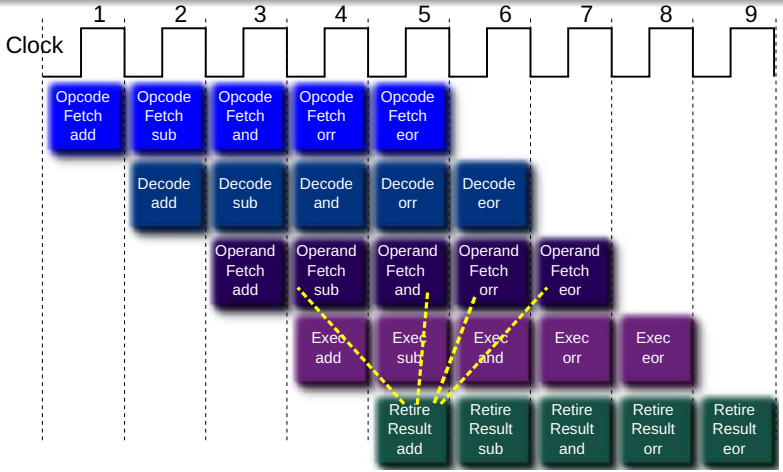
- Se producen cuando por efecto del pipeline, una instrucción requiere de un dato antes de que este esté disponible por efecto de la secuencia lógica prevista en el programa.
- Consideremos el código que figura al pié. Corresponde a un procesador ARM.
- Se observa dependencias para el Registro R1. Hasta que la instrucción ***add*** no complete su operación, R1 no tiene un valor válido. Por lo tanto no puede continuar aplicándolo en las restantes que lo utilizan.
- La situación en el pipeline se muestra en el próximo slide.

Obstáculos de datos

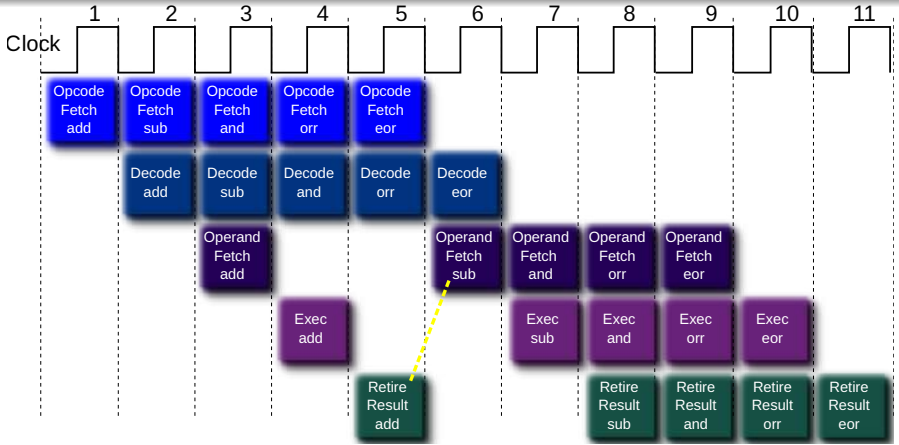
- Se producen cuando por efecto del pipeline, una instrucción requiere de un dato antes de que este esté disponible por efecto de la secuencia lógica prevista en el programa.
- Consideremos el código que figura al pié. Corresponde a un procesador ARM.
- Se observa dependencias para el Registro R1. Hasta que la instrucción ***add*** no complete su operación, R1 no tiene un valor válido. Por lo tanto no puede continuar aplicándolo en las restantes que lo utilizan.
- La situación en el pipeline se muestra en el próximo slide.

```
add   R1,  R2, R3   ; R1 es el destino de esta operación
sub   R4,  R1, R5   ; R1 es operando pero aún no fue escrito el resultado
and   R6,  R1, R7   ;
orr   R8,  R1, R9   ; Se ejecutan finalizada la anterior
eor   R10, R1, R11  ;
```

Obstáculos de Datos - Conflicto de recursos

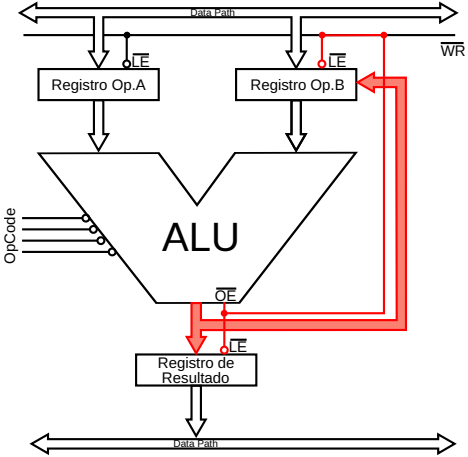


Obstáculos de Datos - Efecto en CPI

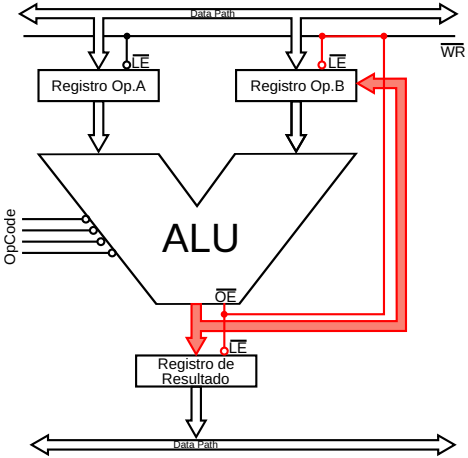


- Por cada Dependencia, pospone una operación. $CPI = CPI + n$, siendo n la distancia entre las etapas del pipeline que requieren el mismo dato.

Solución a Obstáculos de Datos: Forwarding

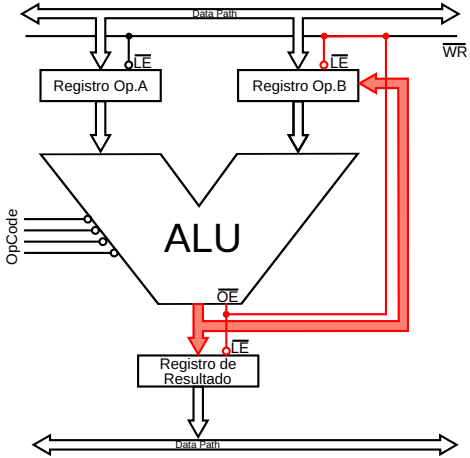


Solución a Obstáculos de Datos: Forwarding



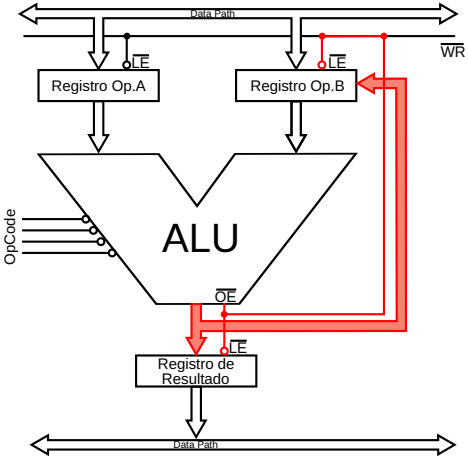
- Ni bien el resultado está disponible se lo realimenta a la entrada de la Unidad de Ejecución (ALU, FPU, etc.).

Solución a Obstáculos de Datos: Forwarding



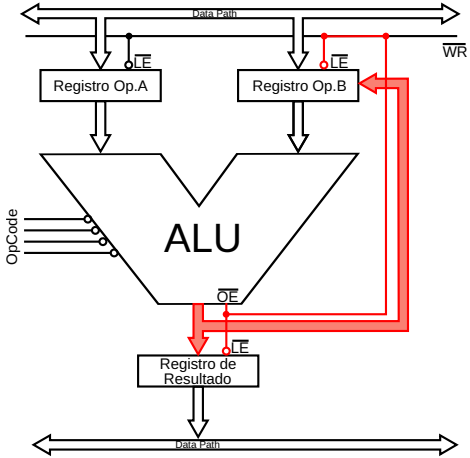
- Ni bien el resultado está disponible se lo realimenta a la entrada de la Unidad de Ejecución (ALU, FPU, etc.).
- Así, la etapa que lo requiere lo dispone en el mismo ciclo de clock en que se escribirá en el operando destino.

Solución a Obstáculos de Datos: Forwarding



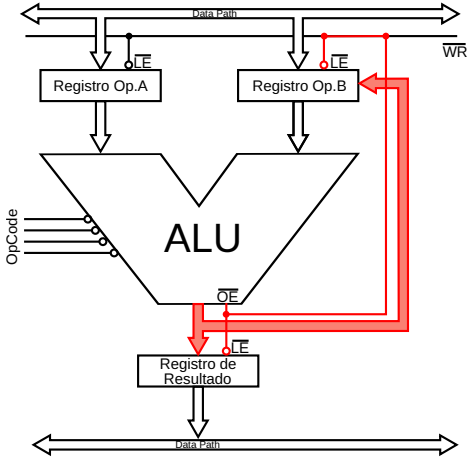
- Ni bien el resultado está disponible se lo realimenta a la entrada de la Unidad de Ejecución (ALU, FPU, etc.).
- Así, la etapa que lo requiere lo dispone en el mismo ciclo de clock en que se escribirá en el operando destino.
- Esto permite ahorrar el tiempo de escritura en el operando destino (No espera por el operando destino).

Solución a Obstáculos de Datos: Forwarding



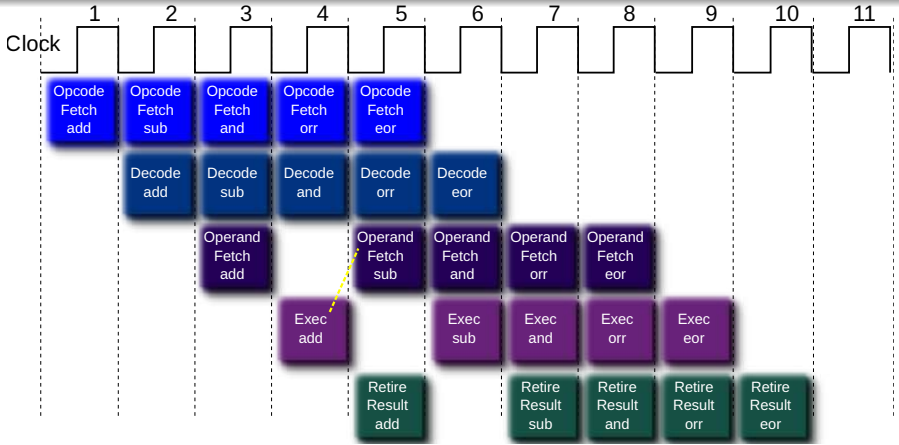
- Ni bien el resultado está disponible se lo realimenta a la entrada de la Unidad de Ejecución (ALU, FPU, etc.).
- Así, la etapa que lo requiere lo dispone en el mismo ciclo de clock en que se escribirá en el operando destino.
- Esto permite ahorrar el tiempo de escritura en el operando destino (No espera por el operando destino).
- Se aplica solamente a las etapas posteriores que quedarían en estado stall.

Solución a Obstáculos de Datos: Forwarding



- Ni bien el resultado está disponible se lo realimenta a la entrada de la Unidad de Ejecución (ALU, FPU, etc.).
- Así, la etapa que lo requiere lo dispone en el mismo ciclo de clock en que se escribirá en el operando destino.
- Esto permite ahorrar el tiempo de escritura en el operando destino (No espera por el operando destino).
- Se aplica solamente a las etapas posteriores que quedarían en estado stall.
- Aquellas etapas que a pesar de la dependencia de datos, no quedarían en estado stall, reciben el resultado cuando este es aplicado en el operando destino.

Obstácualos de Datos - Forwarding



- No resuelve completamente el problema pero reduce el efecto al 50 %.

Algunas veces forwarding no es factible

Algunas veces forwarding no es factible

- Normalmente forwarding aplica a operaciones de ALU denominadas back-to-back, ya que el bypass se efectúa desde la ALU a un registro entero del procesador.

Algunas veces forwarding no es factible

- Normalmente forwarding aplica a operaciones de ALU denominadas back-to-back, ya que el bypass se efectúa desde la ALU a un registro entero del procesador.
- Consideremos ahora esta variante del código anterior.

```
ldr   R1,  [R2]      ;Lectura de memoria no pasa por la ALU.
sub   R4,  R1,  R5   ;R1 no puede ser forwardeado al ingresar.
and   R6,  R1,  R7
orr   R8,  R1,  R9
eor   R10, R1,  R11
```

Algunas veces forwarding no es factible

- Normalmente forwarding aplica a operaciones de ALU denominadas back-to-back, ya que el bypass se efectúa desde la ALU a un registro entero del procesador.
- Consideremos ahora esta variante del código anterior.

```
ldr   R1,  [R2]      ;Lectura de memoria no pasa por la ALU.
sub   R4,  R1,  R5   ;R1 no puede ser forwardeado al ingresar.
and   R6,  R1,  R7
orr   R8,  R1,  R9
eor   R10, R1,  R11
```

- En este caso el dato proveniente de memoria es almacenado en el registro de datos que interfacea con el Bus de datos

Algunas veces forwarding no es factible

- Normalmente forwarding aplica a operaciones de ALU denominadas back-to-back, ya que el bypass se efectúa desde la ALU a un registro entero del procesador.
- Consideremos ahora esta variante del código anterior.

```
ldr   R1,  [R2]      ;Lectura de memoria no pasa por la ALU.
sub   R4,  R1,  R5   ;R1 no puede ser forwardeado al ingresar.
and   R6,  R1,  R7
orr   R8,  R1,  R9
eor   R10, R1,  R11
```

- En este caso el dato proveniente de memoria es almacenado en el registro de datos que interfacea con el Bus de datos
- Desde este registro es enviado directamente al registro destino, en este caso R1.

Algunas veces forwarding no es factible

- Normalmente forwarding aplica a operaciones de ALU denominadas back-to-back, ya que el bypass se efectúa desde la ALU a un registro entero del procesador.
- Consideremos ahora esta variante del código anterior.

```
ldr   R1,  [R2]      ;Lectura de memoria no pasa por la ALU.
sub   R4,  R1,  R5   ;R1 no puede ser forwardeado al ingresar.
and   R6,  R1,  R7
orr   R8,  R1,  R9
eor   R10, R1,  R11
```

- En este caso el dato proveniente de memoria es almacenado en el registro de datos que interfacea con el Bus de datos
- Desde este registro es enviado directamente al registro destino, en este caso R1.
- No puede adelantarse el valor a la siguiente etapa hasta que no se complete su escritura en R1.

Obstáculos de Control

- La máquina de estados de un Pipeline busca instrucciones en forma secuencial.

Obstáculos de Control

- La máquina de estados de un Pipeline busca instrucciones en forma secuencial.
- Un branch es una instrucción de control de flujo que provoca una discontinuidad en el flujo de ejecución.

Obstáculos de Control

- La máquina de estados de un Pipeline busca instrucciones en forma secuencial.
- Un branch es una instrucción de control de flujo que provoca una discontinuidad en el flujo de ejecución.
- Un branch es la peor situación en pérdida de performance del Pipeline.

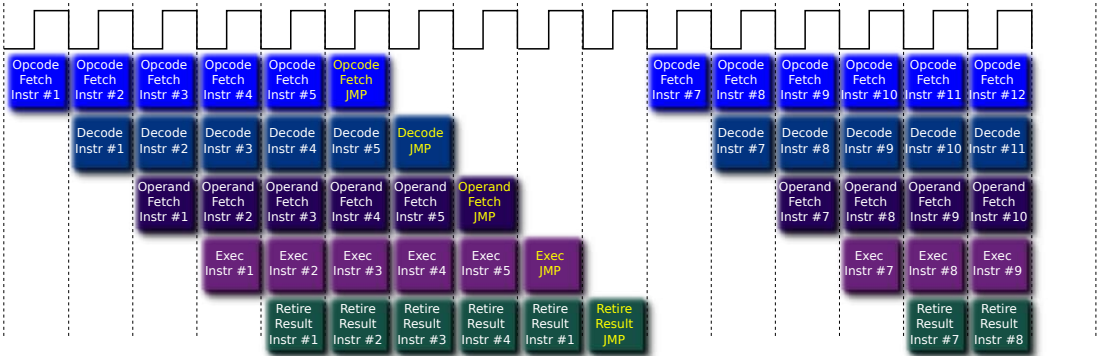
Obstáculos de Control

- La máquina de estados de un Pipeline busca instrucciones en forma secuencial.
- Un branch es una instrucción de control de flujo que provoca una discontinuidad en el flujo de ejecución.
- Un branch es la peor situación en pérdida de performance del Pipeline.
- El branch hace que todo lo que estaba pre procesado deba descartarse, ya que son las instrucciones sucesoras secuenciales al branch en el programa. Al interrumpirse la secuencia de ejecución, ya no son necesarias y deben ser reemplazadas por las instrucciones del target del branch.

Obstáculos de Control

- La máquina de estados de un Pipeline busca instrucciones en forma secuencial.
- Un branch es una instrucción de control de flujo que provoca una discontinuidad en el flujo de ejecución.
- Un branch es la peor situación en pérdida de performance del Pipeline.
- El branch hace que todo lo que estaba pre procesado deba descartarse, ya que son las instrucciones sucesoras secuenciales al branch en el programa. Al interrumpirse la secuencia de ejecución, ya no son necesarias y deben ser reemplazadas por las instrucciones del target del branch.
- El pipeline se vacía. Se requieren ***n -1*** ciclos de clock para obtener el próximo resultado, siendo ***n*** la cantidad de etapas del pipeline. Esto se conoce como **branch penalty**.

”La conspiración de los branches”



saltos, branches, interrupciones, llamadas...

saltos, branches, interrupciones, llamadas...

- En las interrupciones la situación es la misma mostrada en el gráfico del slide anterior para los branches.

saltos, branches, interrupciones, llamadas...

- En las interrupciones la situación es la misma mostrada en el gráfico del slide anterior para los branches.
- En el caso de un branch el principal inconveniente se tiene cuando éste es condicional, ya que es necesario determinar si la condición es true o false.

saltos, branches, interrupciones, llamadas...

- En las interrupciones la situación es la misma mostrada en el gráfico del slide anterior para los branches.
- En el caso de un branch el principal inconveniente se tiene cuando éste es condicional, ya que es necesario determinar si la condición es true o false.
- Si la ondición es true se habla de ***branch taken***. Cambia el registro PC a la dirección de salto, y limpia el pipeline.

saltos, branches, interrupciones, llamadas...

- En las interrupciones la situación es la misma mostrada en el gráfico del slide anterior para los branches.
- En el caso de un branch el principal inconveniente se tiene cuando éste es condicional, ya que es necesario determinar si la condición es true o false.
- Si la ondición es true se habla de ***branch taken***. Cambia el registro PC a la dirección de salto, y limpia el pipeline.
- Si resulta False se habla de ***branch untaken***. El PC que apunta a la instrucción siguiente al branch (la sucesora secuencial) se mantiene, y no se limpia el pipeline.

saltos, branches, interrupciones, llamadas...

- En las interrupciones la situación es la misma mostrada en el gráfico del slide anterior para los branches.
- En el caso de un branch el principal inconveniente se tiene cuando éste es condicional, ya que es necesario determinar si la condición es true o false.
- Si la ondición es true se habla de ***branch taken***. Cambia el registro PC a la dirección de salto, y limpia el pipeline.
- Si resulta False se habla de ***branch untaken***. El PC que apunta a la instrucción siguiente al branch (la sucesora secuencial) se mantiene, y no se limpia el pipeline.
- Esta comprobación requiere llegar hasta la Etapa de ejecución del Pipeline.

saltos, branches, interrupciones, llamadas...

- En las interrupciones la situación es la misma mostrada en el gráfico del slide anterior para los branches.
- En el caso de un branch el principal inconveniente se tiene cuando éste es condicional, ya que es necesario determinar si la condición es true o false.
- Si la ondición es true se habla de ***branch taken***. Cambia el registro PC a la dirección de salto, y limpia el pipeline.
- Si resulta False se habla de ***branch untaken***. El PC que apunta a la instrucción siguiente al branch (la sucesora secuencial) se mantiene, y no se limpia el pipeline.
- Esta comprobación requiere llegar hasta la Etapa de ejecución del Pipeline.
- En el caso de un salto incondicional, o llamada a un procedimiento, en la etapa de decodificación ya se tiene la certeza que se producirá el salto, y solo se pierden menos elementos del Pipeline. Hay impacto de todos modos, pero de menor magnitud.

Efectos de los branches y como neutralizarlos

Efectos de los branches y como neutralizarlos

- Forwarding puede ayudar a disminuir el efecto de los diferentes casos expuestos anteriormente. Sin embargo, no es óptima, ya que solo lograremos disminuir algún ciclo de clock del **branch penalty**, pero siendo su efecto tan significativo, la modesta mejora que proporciona Forwarding termina siendo insuficiente.

Efectos de los branches y como neutralizarlos

- Forwarding puede ayudar a disminuir el efecto de los diferentes casos expuestos anteriormente. Sin embargo, no es óptima, ya que solo lograremos disminuir algún ciclo de clock del **branch penalty**, pero siendo su efecto tan significativo, la modesta mejora que proporciona Forwarding termina siendo insuficiente.

- Para soluciones mas eficientes es necesario recurrir a análisis mas elaborados, que en general tienen en cuenta el comportamiento de los algoritmos y la frecuencia con que lso compiladores emplean instrucciones de salto.

Efectos de los branches y como neutralizarlos

- Forwarding puede ayudar a disminuir el efecto de los diferentes casos expuestos anteriormente. Sin embargo, no es óptima, ya que solo lograremos disminuir algún ciclo de clock del **branch penalty**, pero siendo su efecto tan significativo, la modesta mejora que proporciona Forwarding termina siendo insuficiente.

- Para soluciones mas eficientes es necesario recurrir a análisis mas elaborados, que en general tienen en cuenta el comportamiento de los algoritmos y la frecuencia con que lso compiladores emplean instrucciones de salto.

- Ingresamos así al universo de las unidades de predicción de saltos. Las hay desde muy simples a muy sofisticadas, y tiene que ver no solo con las diferentes generaciones de procesadores, sino también con el tipo de procesador que se tome.

1 Pioneros

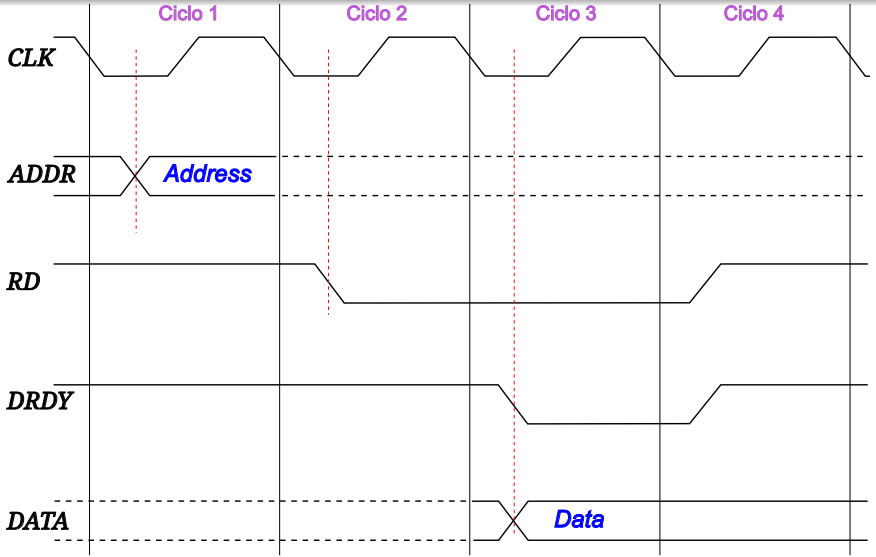
2 Evolución

3 Arquitectura y Organización

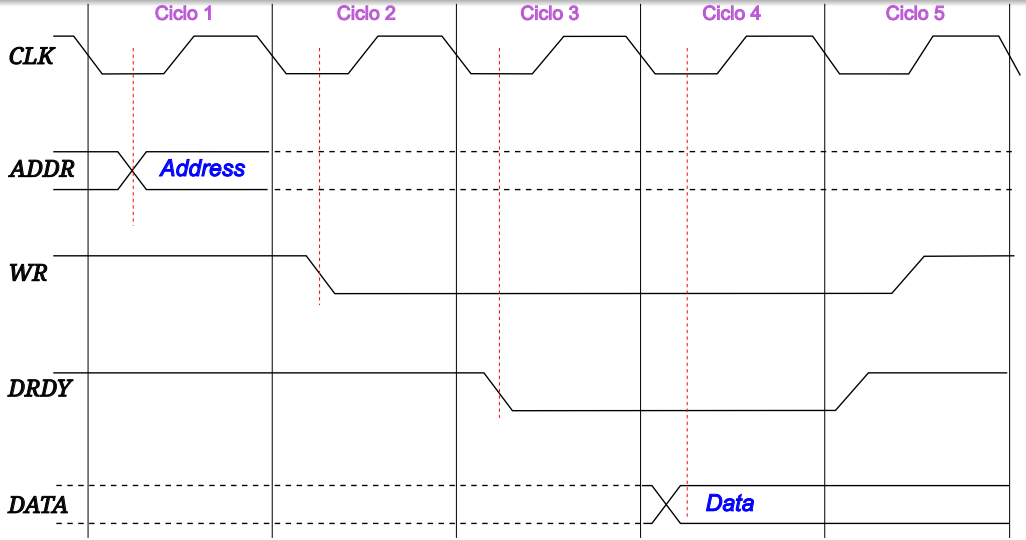
4 Implementando la arquitectura

- El Procesador
- Implementación del Procesador
- Pipeline
- La memoria. El problema del acceso

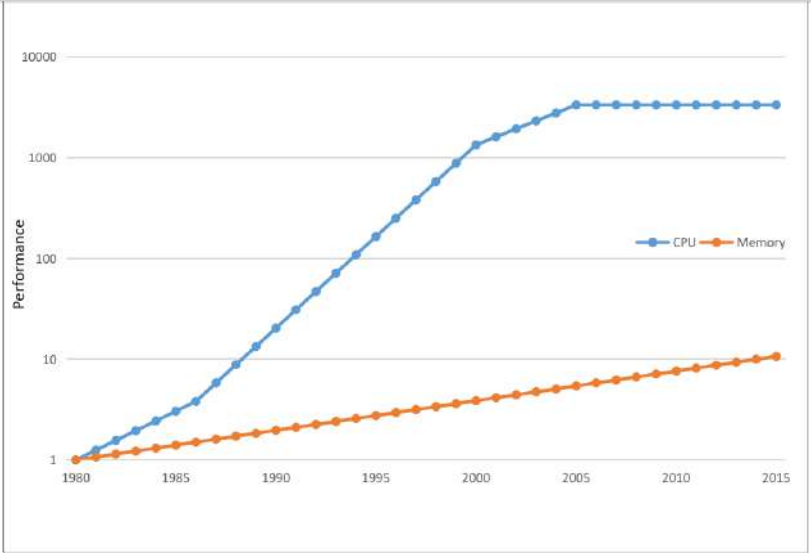
Ciclo de Bus de lectura



Ciclo de Bus de Escritura



Performace Relativa Memoria DRAM vs. CPU



Performace Relativa Memoria DRAM vs. CPU. Notas

- La escala logarítmica clarifica la magnitud del gap en performance.

Performace Relativa Memoria DRAM vs. CPU. Notas

- La escala logarítmica clarifica la magnitud del gap en performance.
- La progresión de memoria es a razón de 1.07 por año. Es constante, y parte de un baseline de 64 KiB en 1980. Terminan en un incremento de aproximadamente x10.

Performace Relativa Memoria DRAM vs. CPU. Notas

- La escala logarítmica clarifica la magnitud del gap en performance.
- La progresión de memoria es a razón de 1.07 por año. Es constante, y parte de un baseline de 64 KiB en 1980. Terminan en un incremento de aproximadamente x10.
- Para la CPU la base es de una sola CPU por sistema.

Performace Relativa Memoria DRAM vs. CPU. Notas

- La escala logarítmica clarifica la magnitud del gap en performance.
- La progresión de memoria es a razón de 1.07 por año. Es constante, y parte de un baseline de 64 KiB en 1980. Terminan en un incremento de aproximadamente x10.
- Para la CPU la base es de una sola CPU por sistema.
- El crecimiento tiene tramos diferentes: Hasta 1986 1,25 por año, luego hasta 2000 1.52 por año, entre 2000, y 2005 1.2, y a partir de ese año el mejoramiento de la performance entra en una meseta.

Performace Relativa Memoria DRAM vs. CPU. Notas

- La escala logarítmica clarifica la magnitud del gap en performance.
- La progresión de memoria es a razón de 1.07 por año. Es constante, y parte de un baseline de 64 KiB en 1980. Terminan en un incremento de aproximadamente x10.
- Para la CPU la base es de una sola CPU por sistema.
- El crecimiento tiene tramos diferentes: Hasta 1986 1,25 por año, luego hasta 2000 1.52 por año, entre 2000, y 2005 1.2, y a partir de ese año el mejoramiento de la performance entra en una meseta.
- Casualmente en ese año comienzan a presentarse los chips multicore.

Performace Relativa Memoria DRAM vs. CPU. Notas

- La escala logarítmica clarifica la magnitud del gap en performance.
- La progresión de memoria es a razón de 1.07 por año. Es constante, y parte de un baseline de 64 KiB en 1980. Terminan en un incremento de aproximadamente x10.
- Para la CPU la base es de una sola CPU por sistema.
- El crecimiento tiene tramos diferentes: Hasta 1986 1,25 por año, luego hasta 2000 1.52 por año, entre 2000, y 2005 1.2, y a partir de ese año el mejoramiento de la performance entra en una meseta.
- Casualmente en ese año comienzan a presentarse los chips multicore.

Conclusión

Necesitamos una estrategia de diseño para al menos minimizar el efecto de este gap.

Performace Relativa Memoria DRAM vs. CPU. Notas

- La escala logarítmica clarifica la magnitud del gap en performance.
- La progresión de memoria es a razón de 1.07 por año. Es constante, y parte de un baseline de 64 KiB en 1980. Terminan en un incremento de aproximadamente x10.
- Para la CPU la base es de una sola CPU por sistema.
- El crecimiento tiene tramos diferentes: Hasta 1986 1,25 por año, luego hasta 2000 1.52 por año, entre 2000, y 2005 1.2, y a partir de ese año el mejoramiento de la performance entra en una meseta.
- Casualmente en ese año comienzan a presentarse los chips multicore.

Conclusión

Necesitamos una estrategia de diseño para al menos minimizar el efecto de este gap.

Continuará...

¿Preguntas?